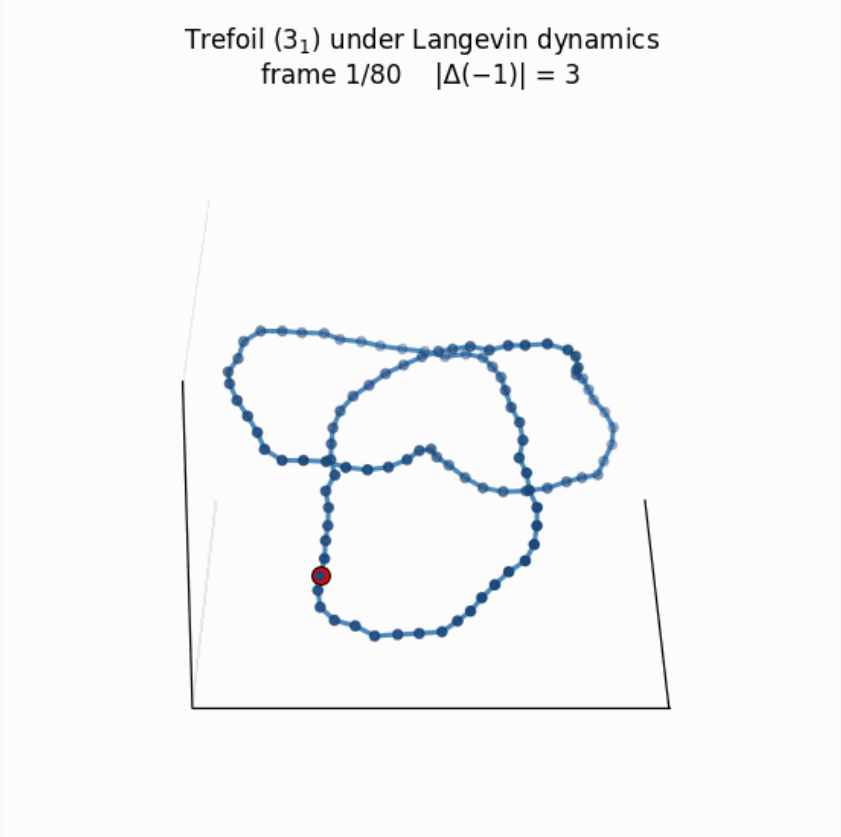
Week 9
LLMs and reasoning

Nebius Academy

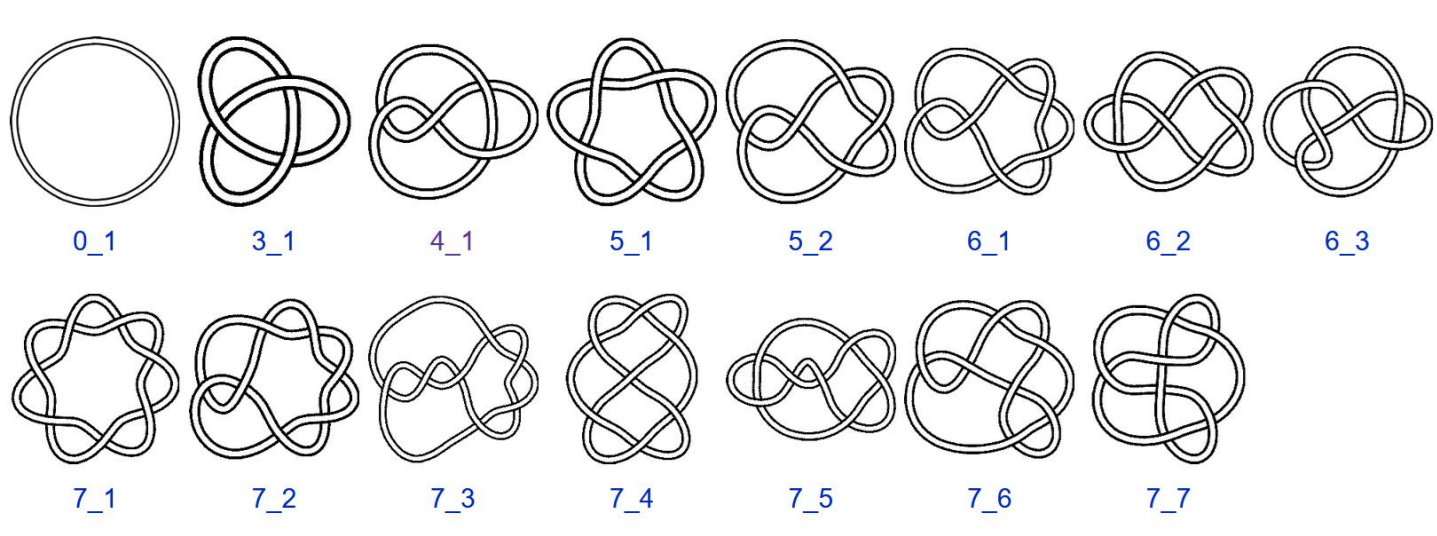
Transformers for knots (surrogate modeling)

Polymers as knots



Knot types

Notation	Common name
0_1	Unknot
3_1	Trefoil
4_1	Figure-eight
5_1	Torus knot
5_2	Three-twist



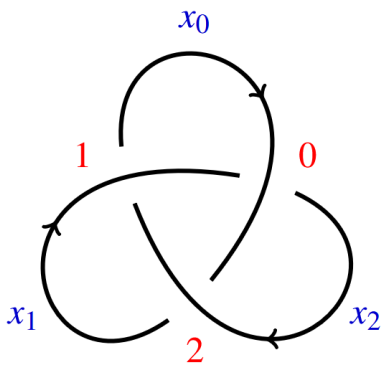
The Alexander polynomial

The equivalence:

$f \doteq g$ if $f(f) = \pm t^m g(t)$

If is a well-defined invariant

Example 5.4. *The Alexander polynomial of the trefoil is* $\Delta_{3_1}(t) = t^2 - t + 1$. To see this we label the arcs and crossings and write down the polynomial colouring equations and the corresponding matrix.



$$0: \; x_0 + tx_1 - tx_0 - x_2 = 0$$
$$1: \; x_1 + tx_2 - tx_1 - x_0 = 0$$
$$2: \; x_2 + tx_0 - tx_2 - x_1 = 0$$

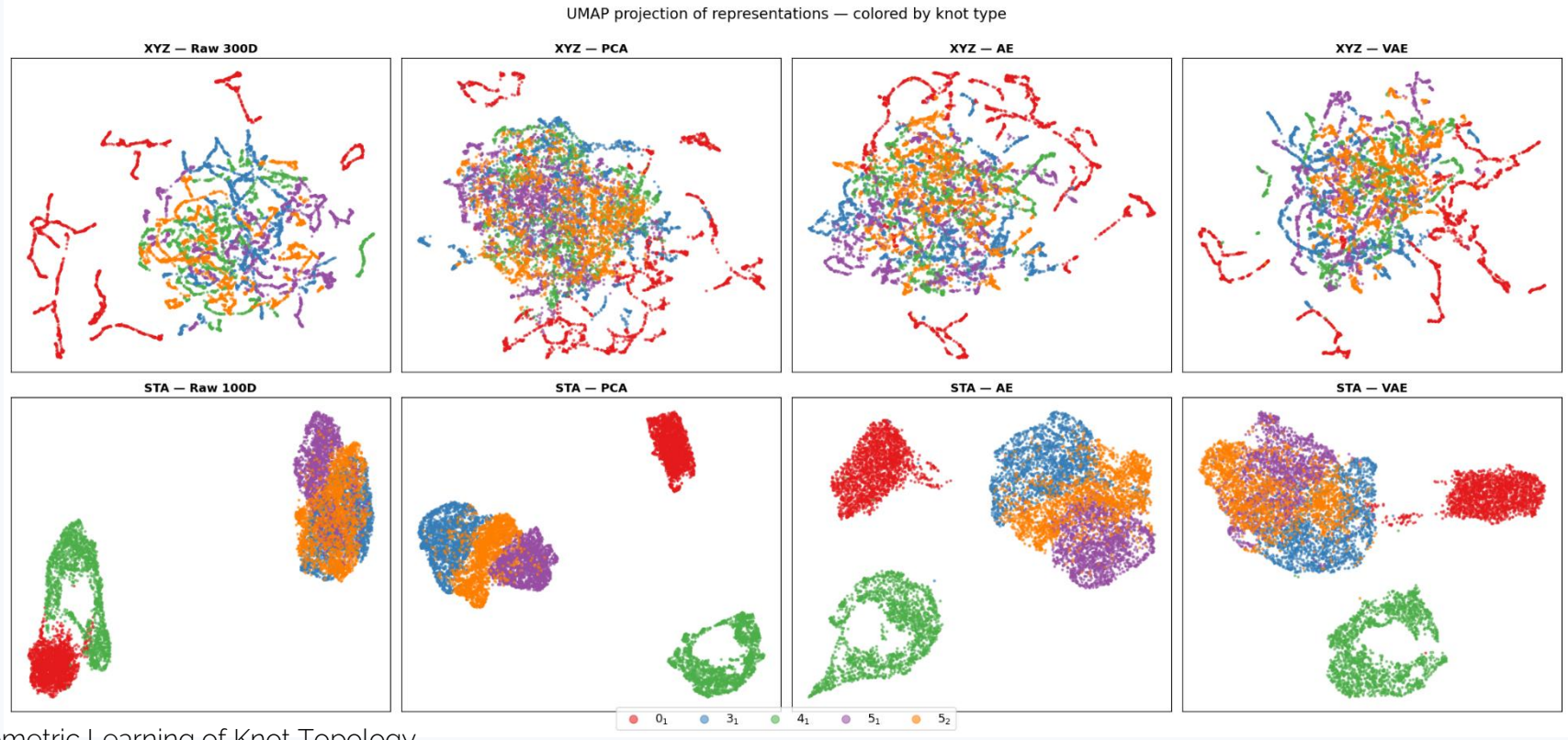
$$\begin{pmatrix} 1-t & t & -1 \\ -1 & 1-t & t \\ t & -1 & 1-t \end{pmatrix}$$

Discard the first row and column of the matrix and compute the determinant to find

$$\Delta_{3_1}(t) = \det \begin{pmatrix} 1-t & t \\ -1 & 1-t \end{pmatrix} = (1-t)^2 + t = t^2 - t + 1$$

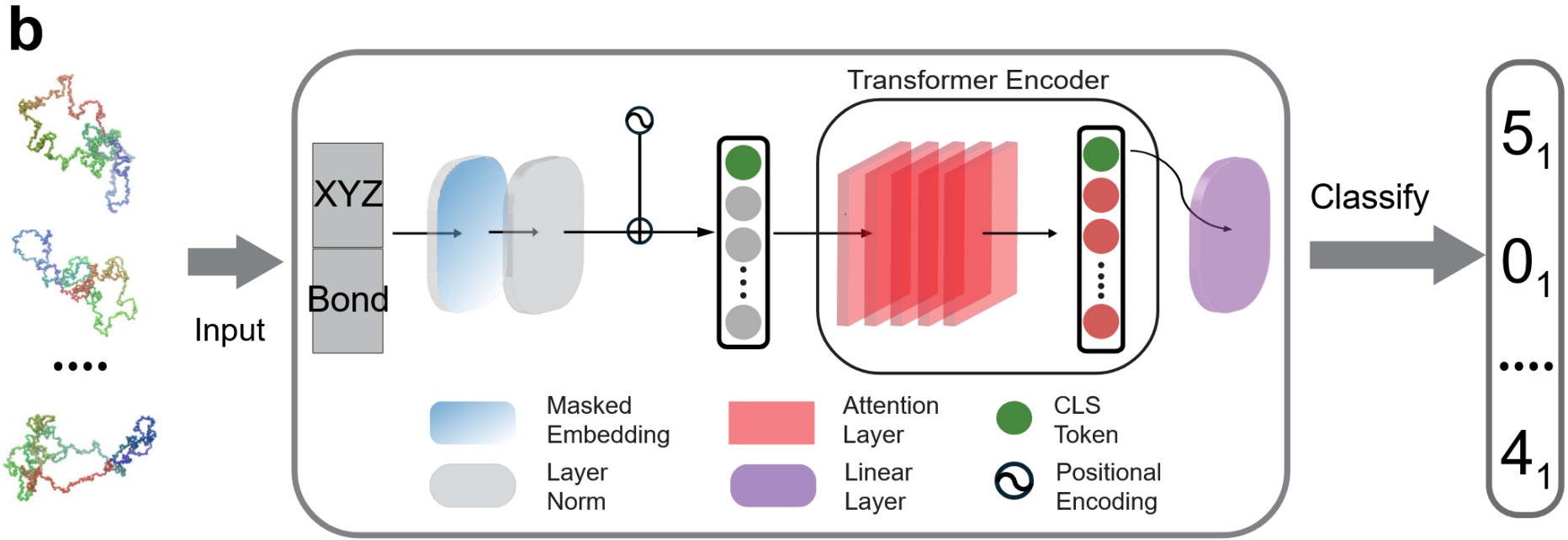
https://homepages.abdn.ac.uk/r.hepworth/pages/files/Knots_Notes.pdf

Knots vs various dimensionality reduction techniques



Geometric Learning of Knot Topology
https://www.researchgate.net/publication/374895179_Geometric_Learning_of_Knot_Topology

Classifying knots with Transformers



Recognizing and generating knotted molecular structures by machine learning
https://arxiv.org/pdf/2501.12780

Classifying knots with Transformers

TABLE I. A comparison between the existing models and our new model with Transformer as the architecture. Here, L_p is the persistence length, which describes the bending stiffness of the chain. $L_p = 0$ means no bending energy, i.e., the adjacent three beads are free to bend. The X-marks indicate that the model is unable to predict such length without truncation of the polymer; the em-dashes represent the absence of data.

Model	Feature	L_p	$N = 100$	$N = 200$	$N = 300$	$N = 512$	$N = 1000$	$\forall N \leq 1000$
Bi-LSTM (Vandas et al. 2020)	Bond vector	$10a$	99.59%	✗	✗	✗	✗	✗
Bi-LSTM (Braghetto et al. 2023)	Bond vector	**0**	—	—	—	80% [a]	✗	✗
Bi-LSTM (Sleiman et al. 2024)	Segment writhe	$10a$	99.8%	**99.6%**	✗	✗	✗	✗
Transformer (This paper)	Coordinates + Bond vector	**0**	**99.9999%**	99.08%	**99.9986%**	**99.23%**	**97.91%**	**99.39%** [b]

[a] Polymers are partitioned into $N' = 128$.
[b] On test set.

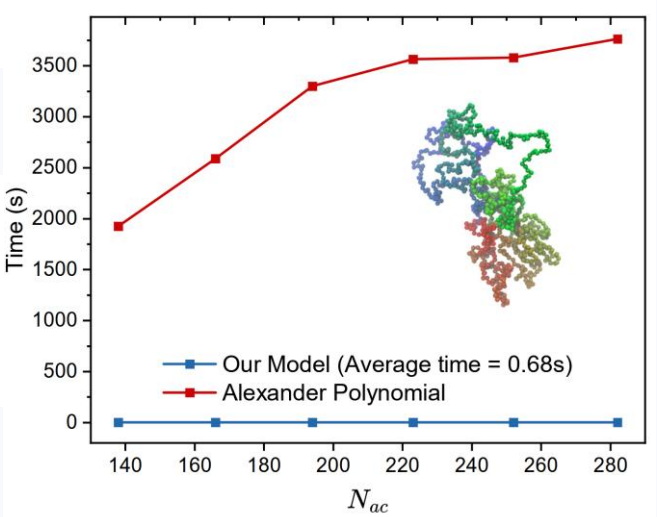


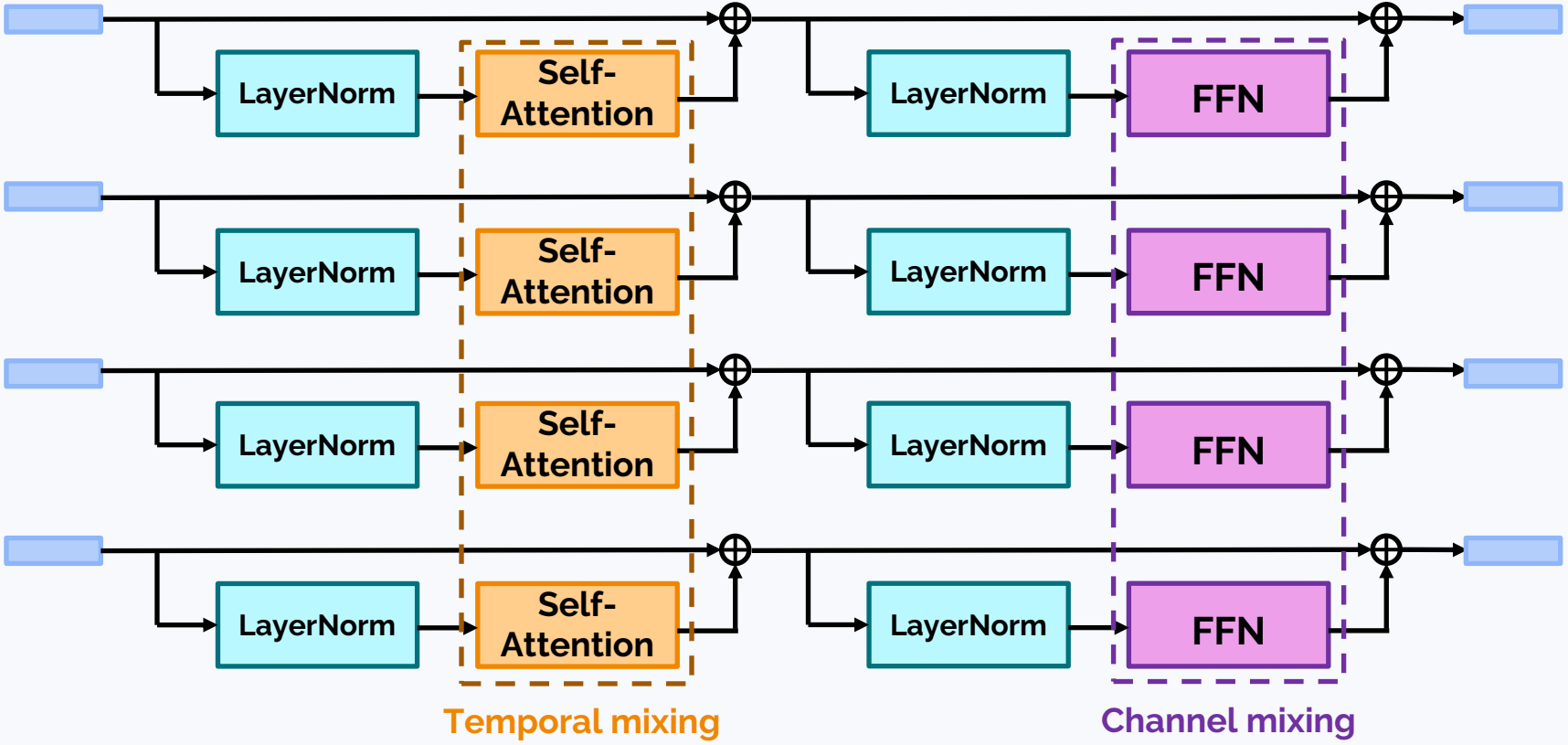
FIG. 3. Comparison of total computational time of identifying 500 polymer knots with high apparent crossing numbers N_{ac} using traditional Alexander Polynomial and our NN model. Neural Network beats traditional Alexander Polynomial in speed. Inset shows a complex trefoil knot with $N_{ac} = 258$ despite having a small minimum crossing number $N_c = 3$.

Recognizing and generating knotted molecular structures by machine learning
https://arxiv.org/pdf/2501.12780

Mixture of Experts

(Almost) classic transformer block in an LLM:

$$z = x + SelfAttention\big(Norm(x)\big) \qquad out = z + FFN\big(Norm(z)\big)$$



Mixture of Experts (MoE)

https://huggingface.co/blog/moe

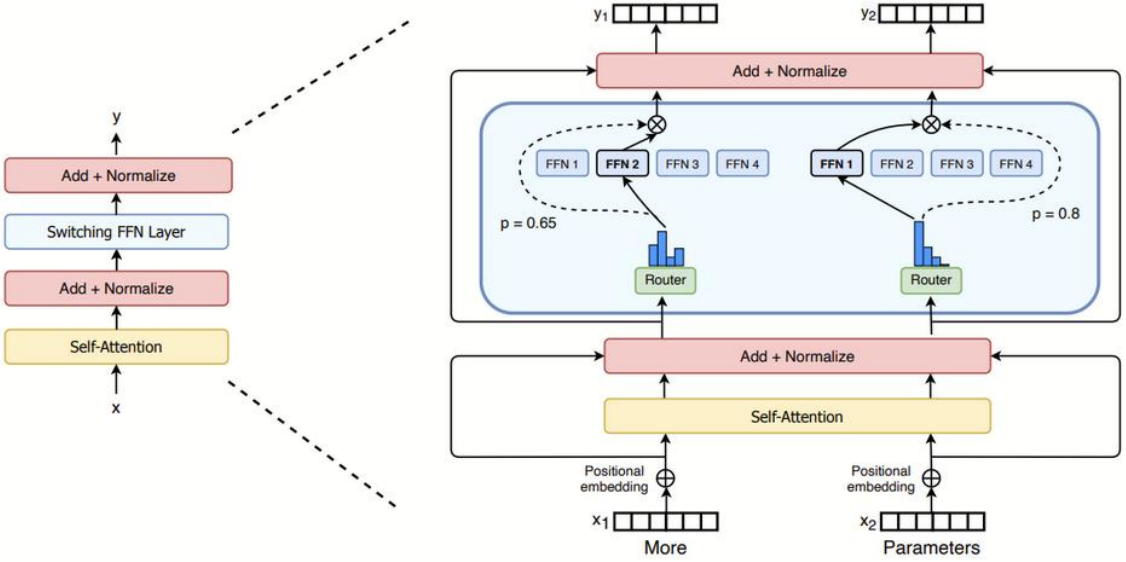


Figure 2: Illustration of a Switch Transformer encoder block. We replace the dense feed forward network (FFN) layer present in the Transformer with a sparse Switch FFN layer (light blue). The layer operates independently on the tokens in the sequence. We diagram two tokens (x_1 = "More" and x_2 = "Parameters" below) being routed (solid lines) across four FFN experts, where the router independently routes each token. The switch FFN layer returns the output of the selected FFN multiplied by the router gate value (dotted-line).

Mixture of Experts (MoE)

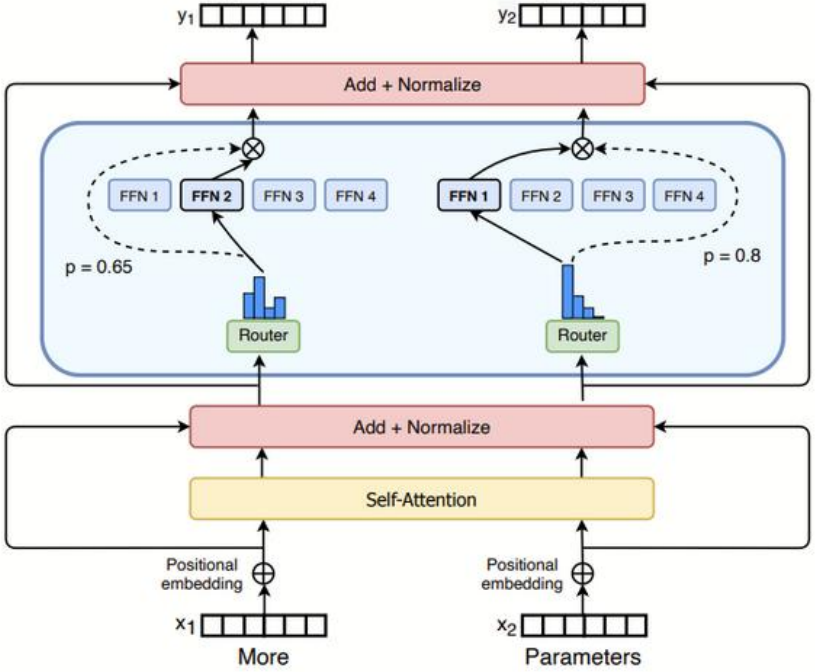
https://huggingface.co/blog/moe

For each expert i:

- $H(x)_i = \left(x \cdot W_g\right)_i +$

$+ \mathcal{N}(0,1) \cdot \text{SoftPlus}\left(x \cdot W_{noise}\right)_i,$

where $\text{SoftPlus}(z) = \log(1 + \exp(z))$



Mixture of Experts (MoE)

https://huggingface.co/blog/moe

For each expert i:

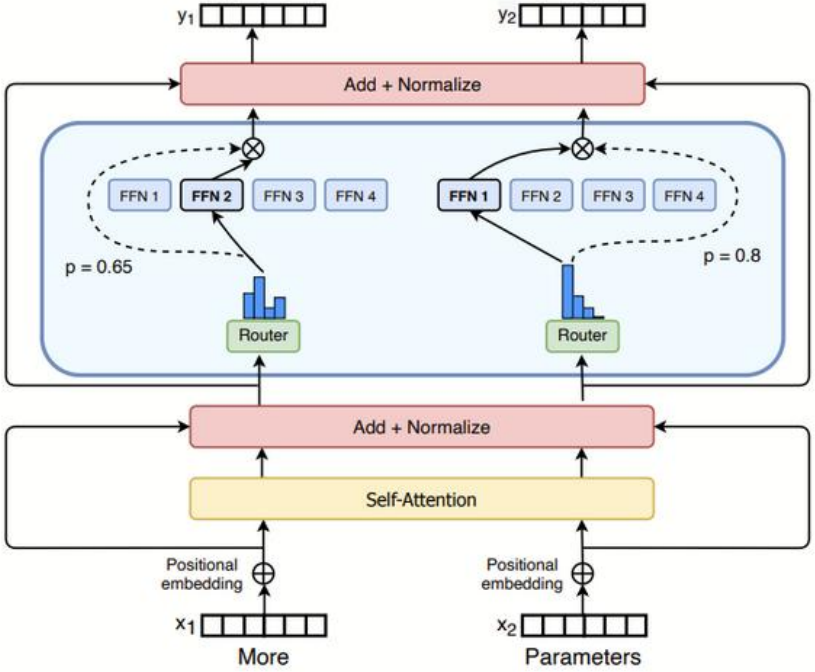
- $H(x)_i = \left(x \cdot W_g\right)_i +$

$+ \mathcal{N}(0,1) \cdot \text{SoftPlus}\left(x \cdot W_{noise}\right)_i,$

where $\text{SoftPlus}(z) = \log(1 + \exp(z))$

- $\text{KeepTopK}(H(x), k))_i =$

$$\begin{cases} H(x)_i, \text{if } H(x)_i \text{ in top } k \text{ of } H(x) \\ -\infty, \text{otherwise} \end{cases}$$



Mixture of Experts (MoE)

https://huggingface.co/blog/moe

For each expert i:

- $H(x)_i = \left(x \cdot W_g\right)_i +$

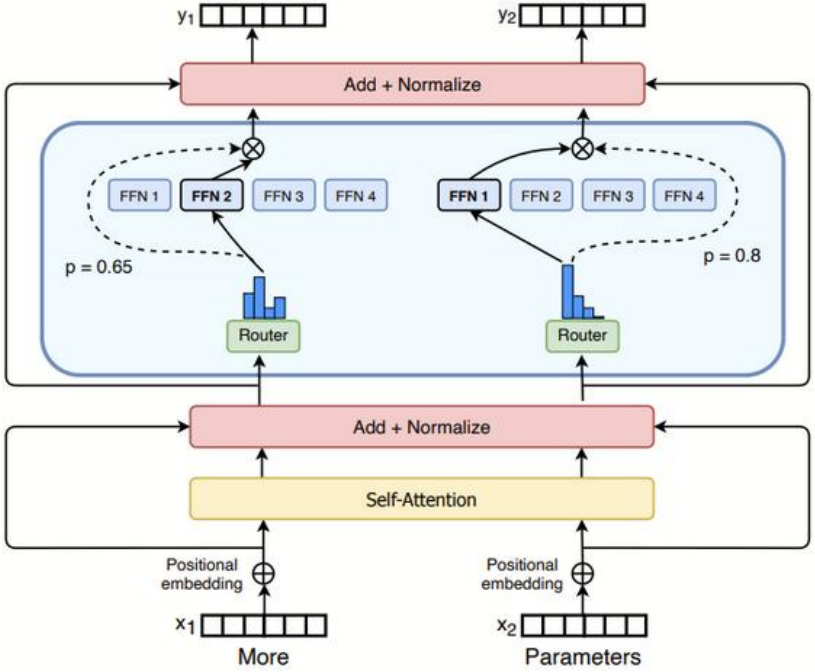
$+ \mathcal{N}(0,1) \cdot \text{SoftPlus}\left(x \cdot W_{noise}\right)_i,$

where $\text{SoftPlus}(z) = \log(1 + \exp(z))$

- $\text{KeepTopK}(H(x), k))_i =$

$$\begin{cases} H(x)_i, \text{if } H(x)_i \text{ in top } k \text{ of } H(x) \\ -\infty, \text{otherwise} \end{cases}$$

- $G(x) = \text{Softmax}\big(\text{KeepTopK}(H(x), k)\big)$



Mixture of Experts (MoE)

https://huggingface.co/blog/moe

For each expert i:

- $H(x)_i = \left(x \cdot W_g\right)_i +$

$+ \mathcal{N}(0,1) \cdot \text{SoftPlus}\left(x \cdot W_{noise}\right)_i$,

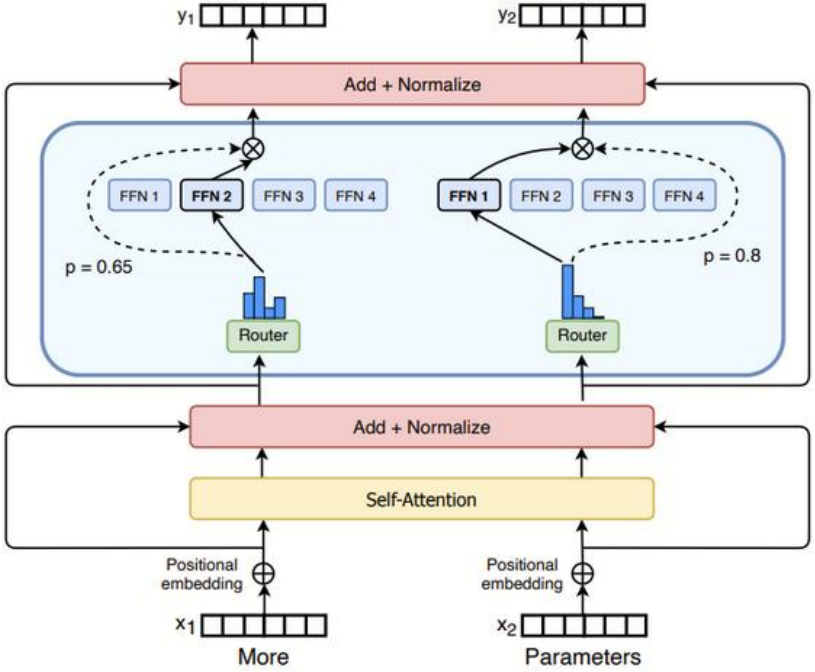
where $\text{SoftPlus}(z) = \log(1 + \exp(z))$

- $\text{KeepTopK}(H(x), k))_i =$

$\begin{cases} H(x)_i, \text{if } H(x)_i \text{ in top } k \text{ of } H(x) \\ -\infty, \text{otherwise} \end{cases}$

- $G(x) = \text{Softmax}\big(\text{KeepTopK}(H(x), k)\big)$

- $y = \sum_i G(x)_i \, E_i(x)$



LLM and API basics (colab)

Reasoning

Chain-of-thought reasoning

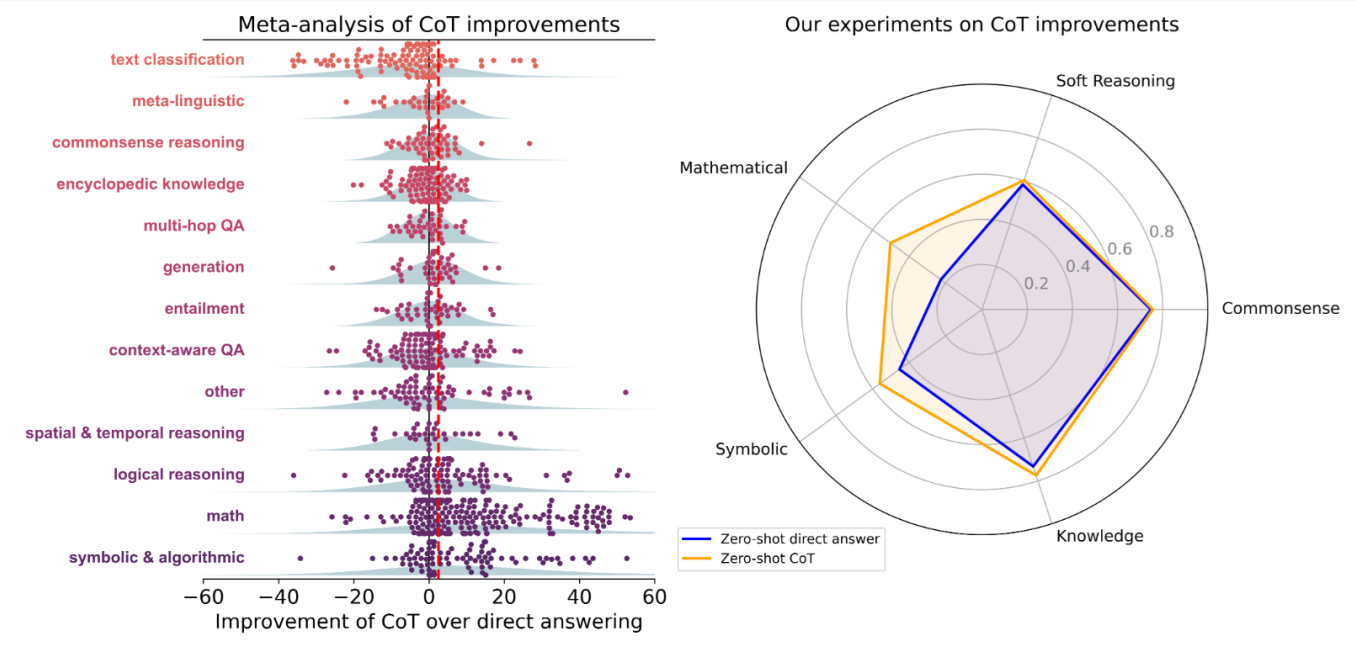
- Zero-shot strategy – just give an answer
- CoT strategy – “Hold your breath and solve the problem step by step”

  It was considered one of the “**emerging capabilities**”

Is reasoning always useful?

To CoT or not to CoT?
https://arxiv.org/abs/2409.12183

The shorter answer is **no**
Useful for symbolic computations
Not useful for knowledge- and commonsense-related tasks



Is reasoning always useful?

Mind your Step (by Step)
https://arxiv.org/pdf/2410.21333

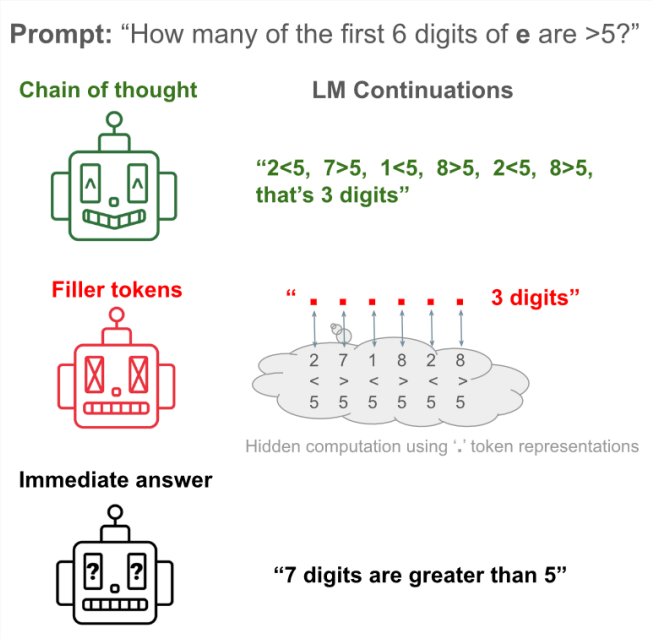
The shorter answer is **no**
Useful for symbolic computations
Not useful for some visual tasks including facial recognition

Table 2. CoT reduces performance for facial recognition across all six LMMs. Random performance is 20%.

	Zero-shot	CoT	decrease (absolute)	decrease (relative)	p-value
GPT-4o	64.00%	51.20%	12.80%	20.00%	< 0.01
Claude 3 Opus	44.00%	29.60%	14.40%	32.73%	< 0.0001
Claude 3.5 Sonnet	97.80%	94.80%	3.00%	3.07%	< 0.05
Gemini 1.5 Pro	66.00%	54.60%	11.40%	17.27%	< 0.05
InternVL2 26B	9.20%	6.00%	3.20%	34.78%	< 0.05
InternVL2 Llama3 76B	15.77%	13.77%	2.00%	12.68%	0.44

Are they really reasoning?!

Do we need words for reasoning?



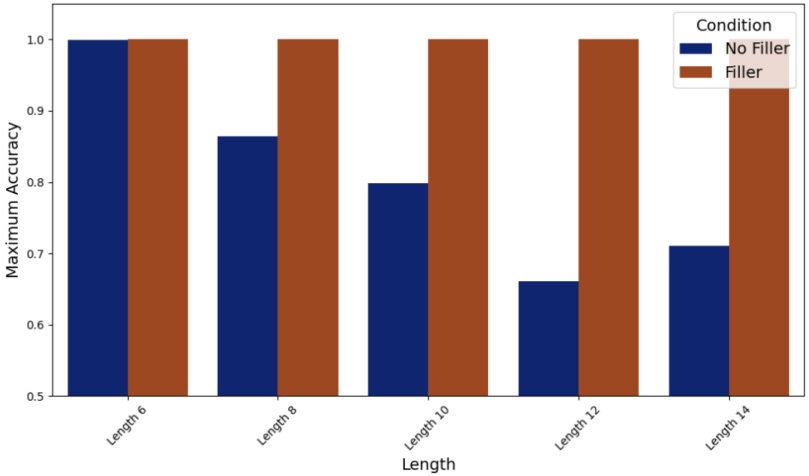
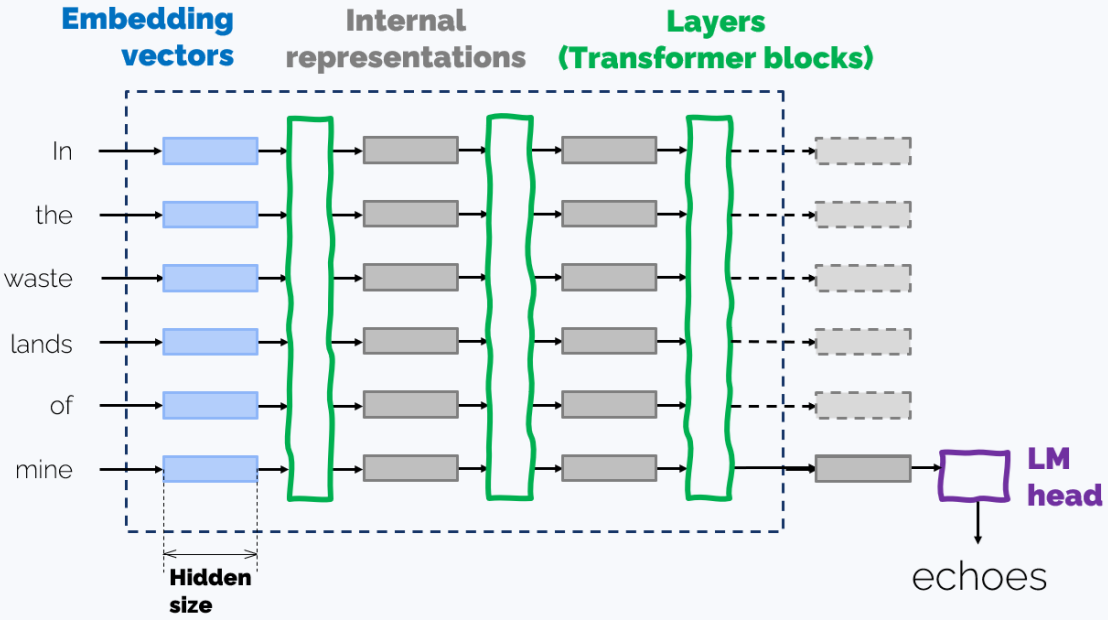


Figure 2: The performance gap between a transformer, Llama 34M, with and without filler tokens increases with 3SUM problem length up to length 12, showing that filler tokens reliably provide an advantage for sufficiently complex 3SUM problems. The no-filler models were trained for 5× the number of steps.

Let's reason dot by dot https://arxiv.org/pdf/2404.15758

Reasoning with linear algebra only

Let's recall the LLM architecture



Reasoning with linear algebra only

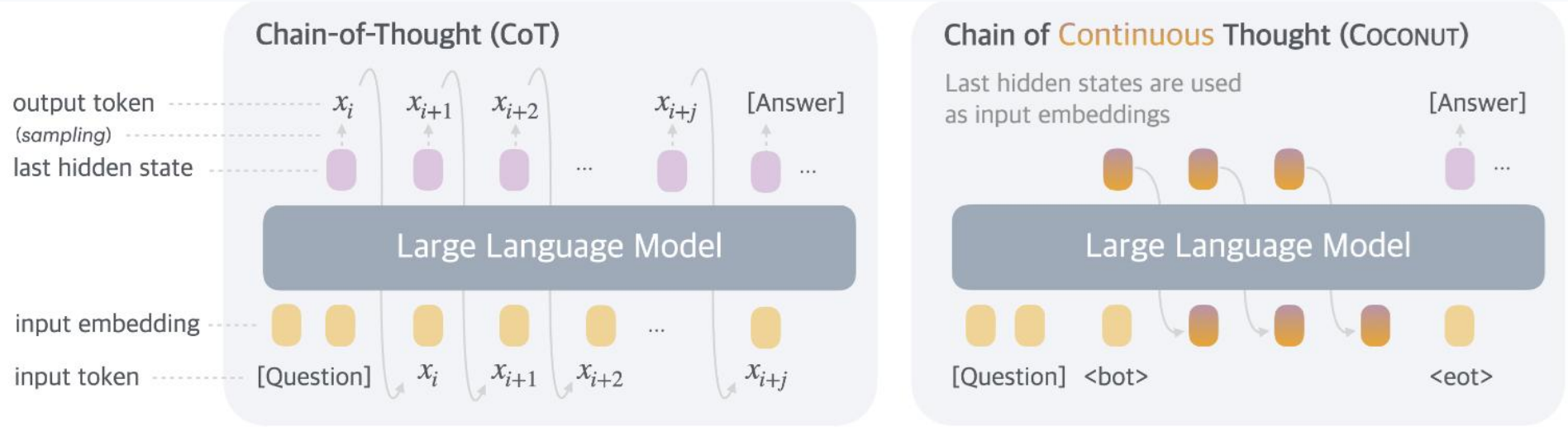


Figure 1 A comparison of Chain of Continuous Thought (COCONUT) with Chain-of-Thought (CoT). In CoT, the model generates the reasoning process as a word token sequence (e.g., $[x_i, x_{i+1}, ..., x_{i+j}]$ in the figure). COCONUT regards the last hidden state as a representation of the reasoning state (termed "continuous thought"), and directly uses it as the next input embedding. This allows the LLM to reason in an unrestricted latent space instead of a language space.

Training Large Language Models to Reason in a Continuous Latent Space https://arxiv.org/pdf/2412.06769

Reasoning with linear algebra only

Method	GSM8k		ProntoQA		ProsQA	
	Acc. (%)	# Tokens	Acc. (%)	# Tokens	Acc. (%)	# Tokens
CoT	42.9 ±0.2	25.0	98.8 ±0.8	92.5	77.5 ±1.9	49.4
No-CoT	16.5 ±0.5	2.2	93.8 ±0.7	3.0	76.7 ±1.0	8.2
iCoT	30.0*	2.2	99.8 ±0.3	3.0	98.2 ±0.3	8.2
Pause Token	16.4 ±1.8	2.2	77.7 ±21.0	3.0	75.9 ±0.7	8.2
COCONUT (Ours)	34.1 ±1.5	8.2	99.8 ±0.2	9.0	97.0 ±0.3	14.2
- *w/o curriculum*	14.4 ±0.8	8.2	52.4 ±0.4	9.0	76.1 ±0.2	14.2
- *w/o thought*	21.6 ±0.5	2.3	99.9 ±0.1	3.0	95.5 ±1.1	8.2
- *pause as thought*	24.1 ±0.7	2.2	100.0 ±0.1	3.0	96.6 ±0.8	8.2

Table 1 Results on three datasets: GSM8l, ProntoQA and ProsQA. Higher accuracy indicates stronger reasoning ability, while generating fewer tokens indicates better efficiency. *The result is from Deng et al. (2024).

Training Large Language Models to Reason in a Continuous Latent Space https://arxiv.org/pdf/2412.06769

A touch of curiosity: latent visual reasoning

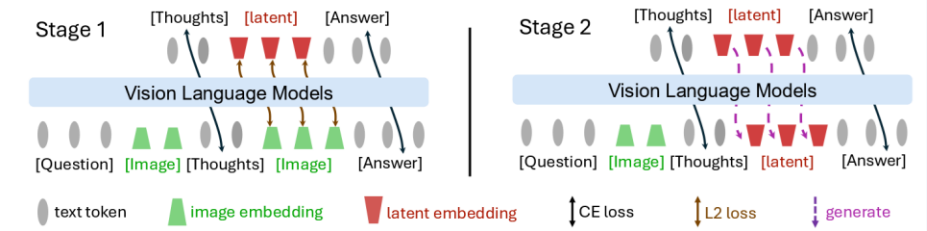


Figure 2: **Pipeline of Mirage Framework.** Stage 1 jointly supervises text and latent visual tokens, grounding the latter in the visual subspace; Stage 2 drops the latent supervision, anchoring the grounded latent tokens for subsequent text generation.

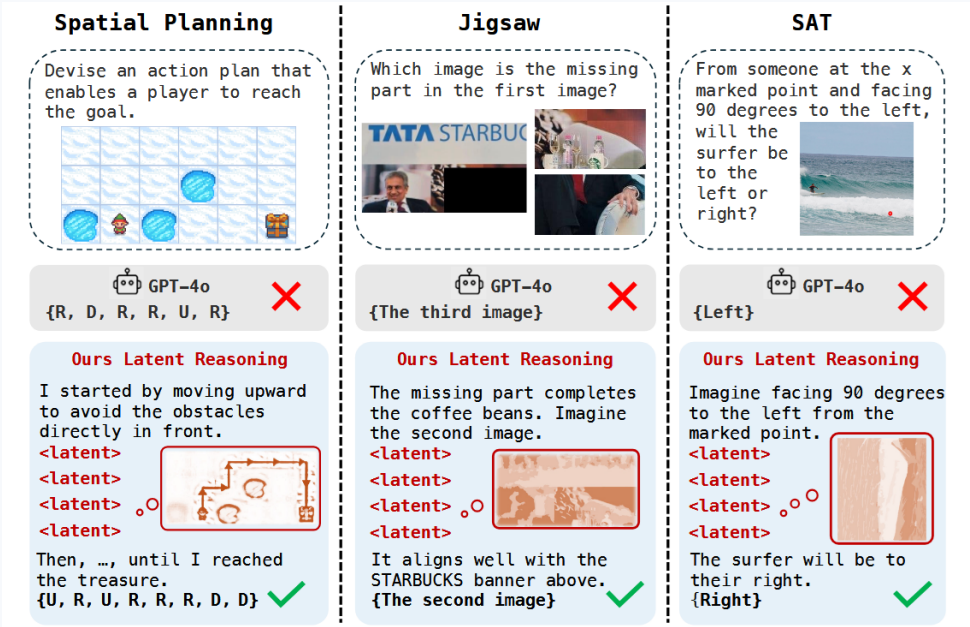


Figure 1: **Multimodal Reasoning Examples.** Mirage interleaves **latent visual tokens**, which represent compact imagery visual features, with explicit text tokens to solve diverse spatial reasoning multimodal tasks, boosting the reasoning performance without the full pixel-level image generation.

Machine Mental Imagery: Empower Multimodal Reasoning with Latent Visual Tokens
https://www.arxiv.org/pdf/2506.17218

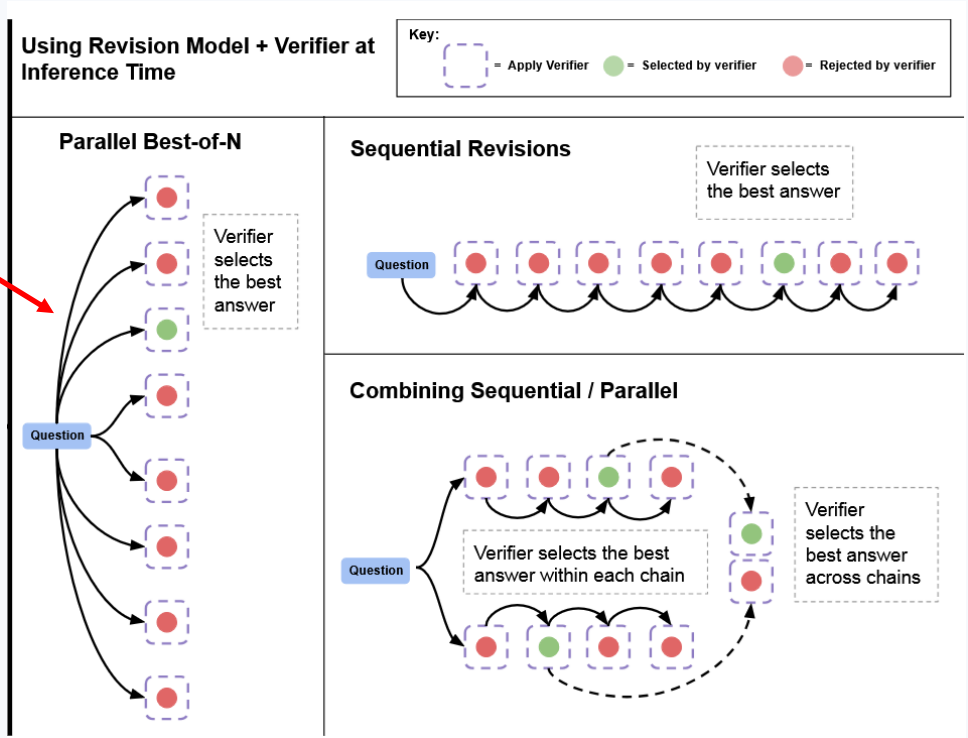
Inference-time compute

Parallel vs sequential orchestration

Self
consistency

N=64
is a normal way
of dealing
with benchmarks :)

Watch out for
“Pass@64”



Scaling LLM Test-Time Compute Optimally can be More Effective than Scaling Model Parameters
https://arxiv.org/abs/2408.03314

An alternative to self-consistency

Use confidence as weights for self-consistency:

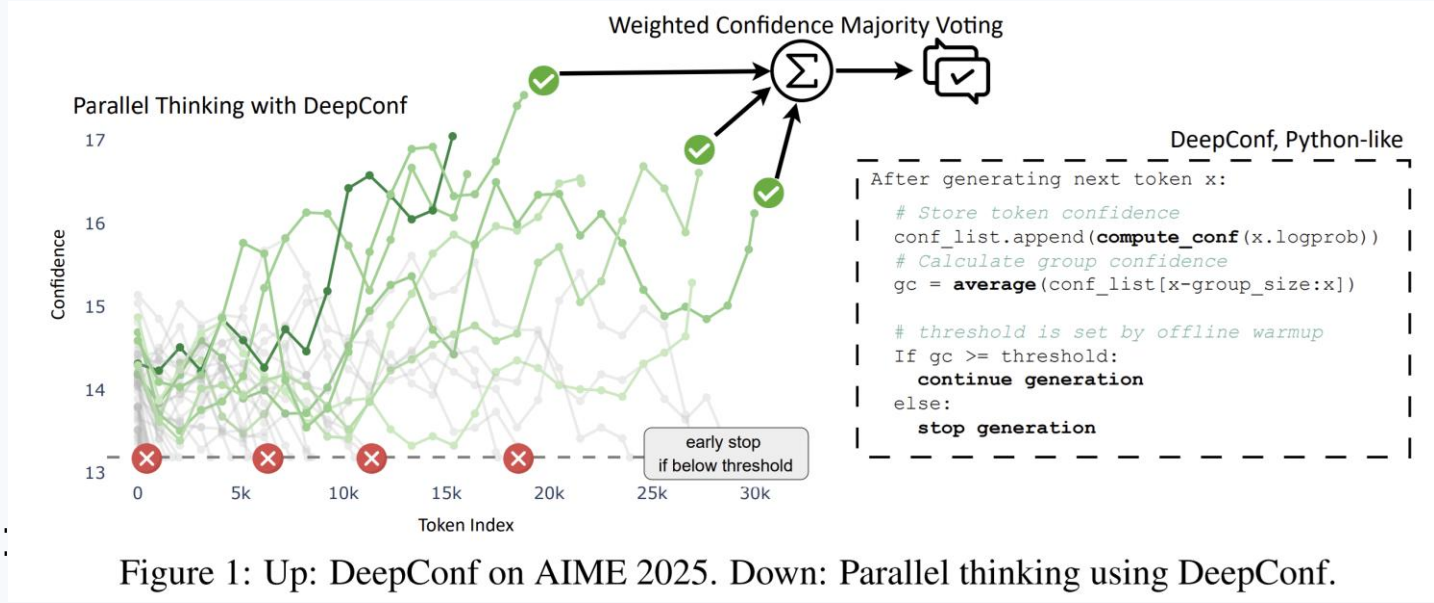
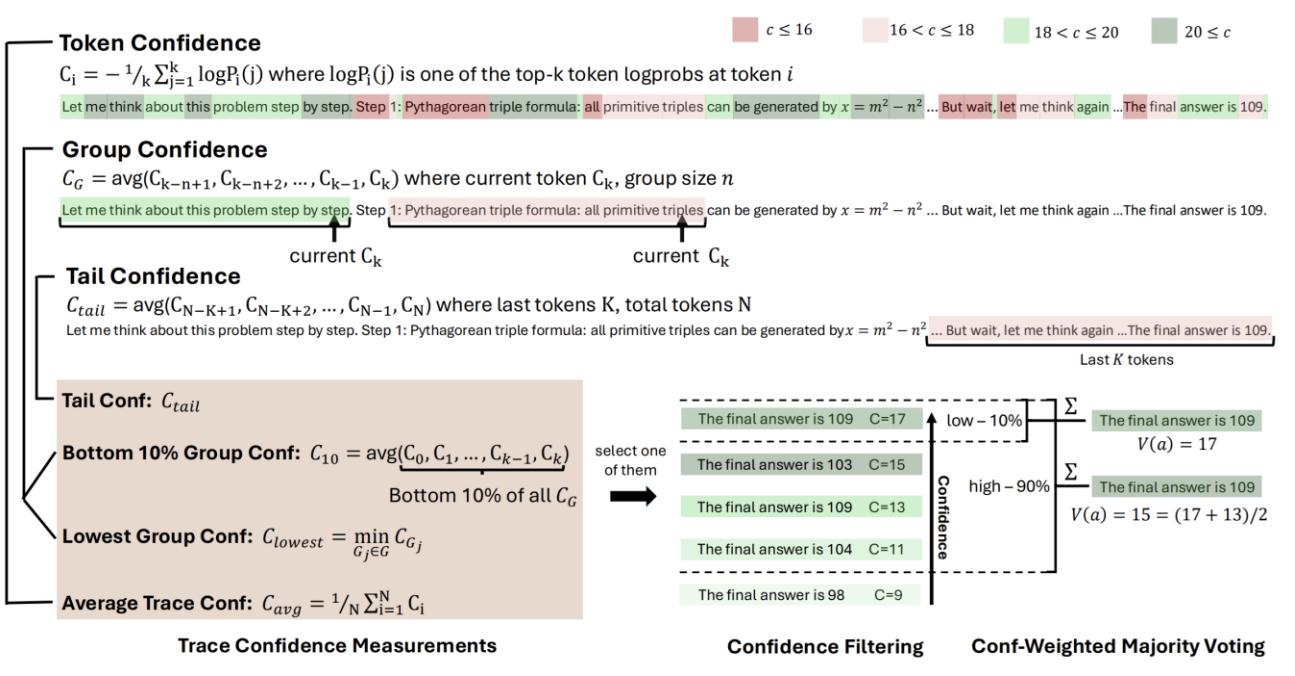


Figure 1: Up: DeepConf on AIME 2025. Down: Parallel thinking using DeepConf.

$$C_i = -\frac{1}{|W|} \sum_{w \in W} \log p(x_i = w \mid x_{1:(i-1)})$$

Deep Think with Confidence
https://arxiv.org/abs/2508.15260

An alternative to self-consistency



$$C_i = -\frac{1}{|W|}\sum_{w \in W} \log p(x_i = w \mid x_{1:(i-1)})$$

Deep Think with Confidence
https://arxiv.org/abs/2508.15260

An alternative to self-consistency

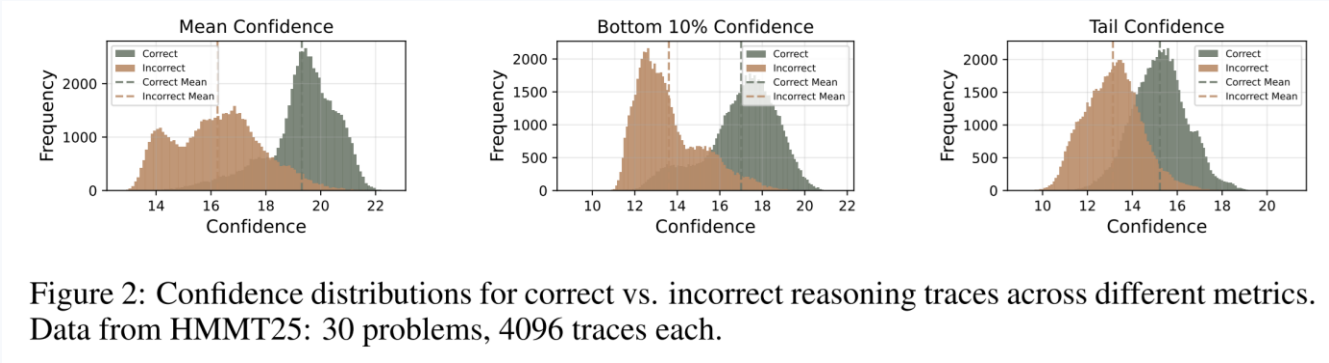
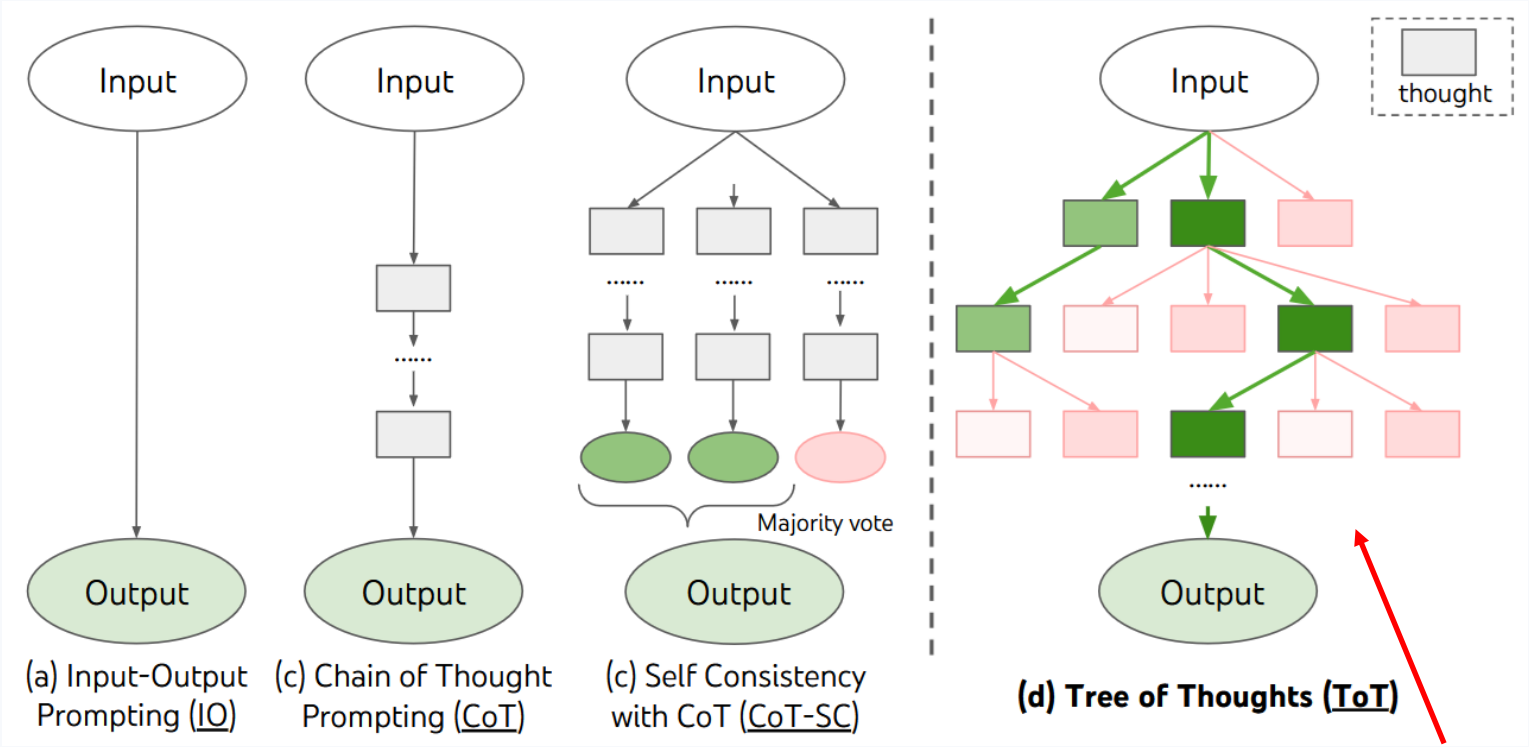


Figure 2: Confidence distributions for correct vs. incorrect reasoning traces across different metrics. Data from HMMT25: 30 problems, 4096 traces each.

Deep Think with Confidence
https://arxiv.org/abs/2508.15260

Non-linear orchestration strategies

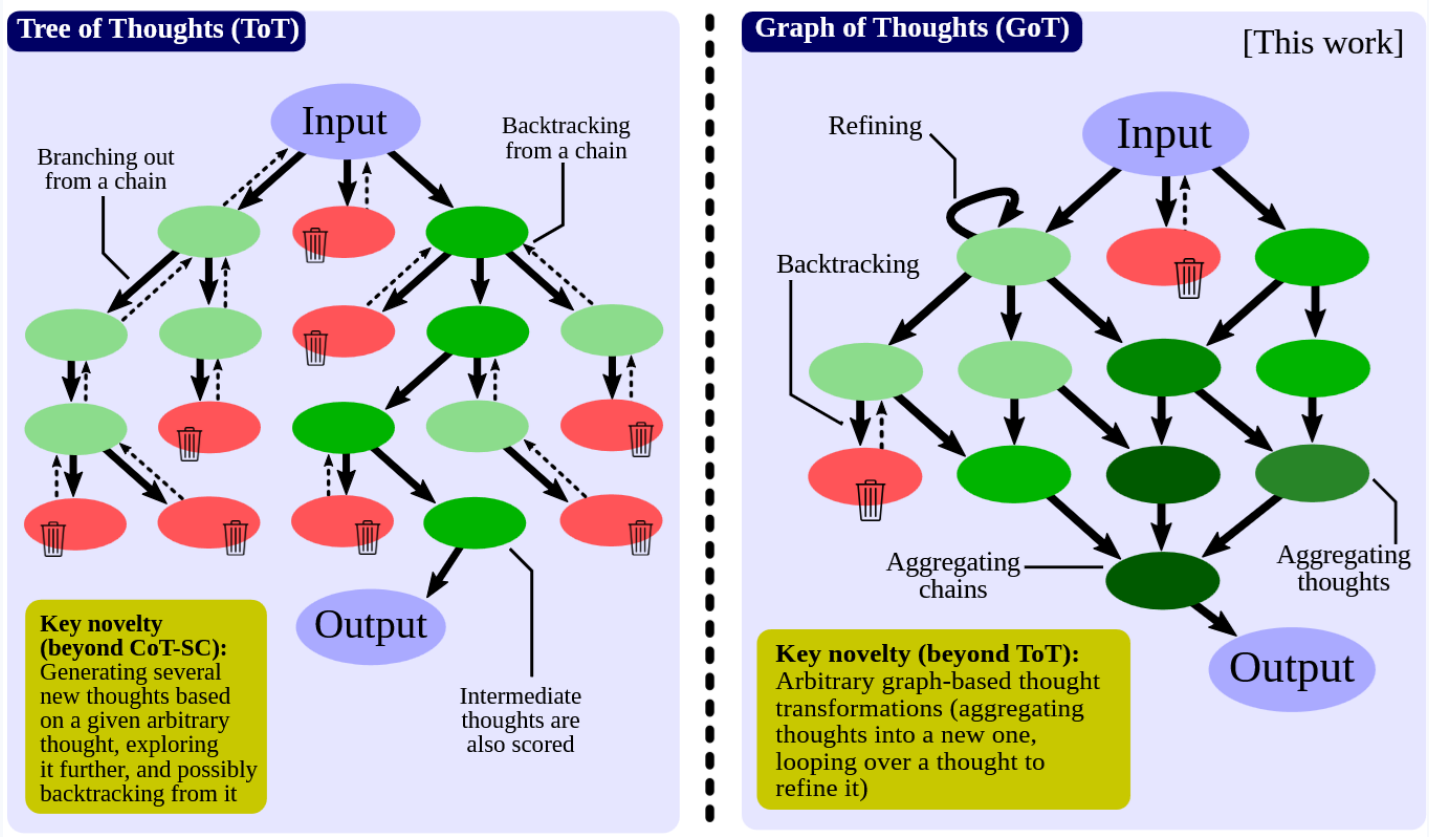


Require

Process Reward Model

Tree of Thoughts: Deliberate Problem Solving with Large Language Models
https://arxiv.org/abs/2305.10601

Non-linear orchestration strategies



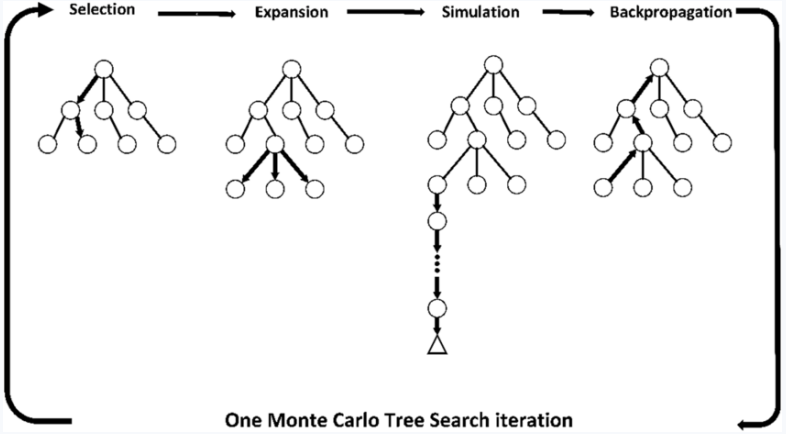
See **Monte Carlo Tree Search (MCTS)**

Monte Carlo Tree Search (MCTS)

Each node is a **partial solution**, storing also:

- n_i: The number of times this node has been visited.
- v_i: The cumulative **reward** from all visits to this node.
- **Possible actions** (i.e., possible next reasoning steps) from this node. At each moment, some actions are **tried**, while others remain **untried**.

It can be used for generating reasoning traces!



https://www.researchgate.net/publication/340287569_Formula-E_race_strategy_development_using_artificial_neural_networks_and_Monte_Carlo_tree_search

Monte Carlo Tree Search (MCTS)

Each node is a **partial solution**, storing also:

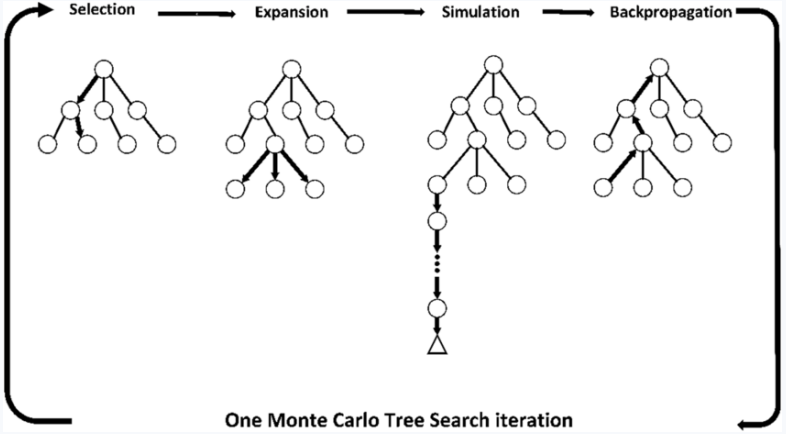
- n_i: The number of times this node has been visited.
- v_i: The cumulative **reward** from all visits to this node.
- **Possible actions** (i.e., possible next reasoning steps) from this node. At each moment, some actions are **tried**, while others remain **untried**.

Four iterative phases:

- **Selection**: Starting from the root, traverse to leaves:
  - If **the current node has untried actions**, select it.
  - Otherwise, choose the **child node** that maximizes the **Upper Confidence Bound (UCB) score**:

$$UCB(i) = \frac{v_i}{n_i} + c\sqrt{\frac{\log n_p}{n_i}},$$

where p is the index of the parent node. UCB tries to balance **exploration** and **exploitation**



https://www.researchgate.net/publication/340287569_Formula-E_race_strategy_development_using_artificial_neural_networks_and_Monte_Carlo_tree_search

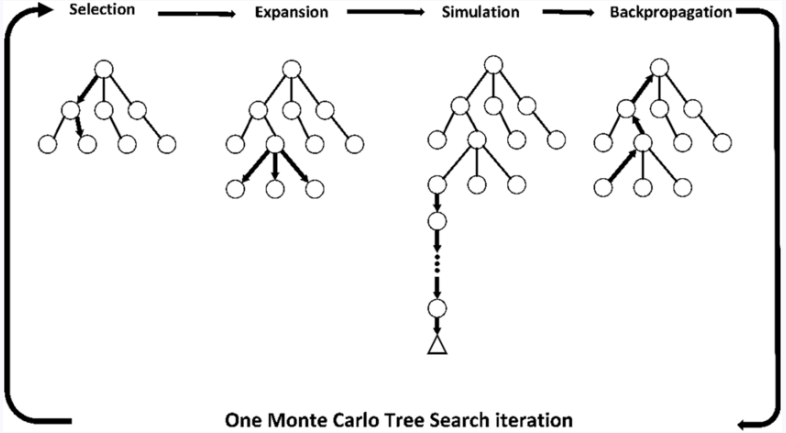
Monte Carlo Tree Search (MCTS)

- n_i: The number of times this node has been visited.
- v_i: The cumulative **reward** from all visits to this node.

Four iterative phases:

- **Selection**: select next node
- **Expansion**: for the selected node, **explore one untried action** (one of the reasoning continuations), which means:

  - If new partial solution is **not terminal** (i.e., does not yet contain the final answer), generate several possible next steps.
  - The *number of generated next steps* is controlled by a hyperparameter (**branching_factor**).
  - The **max_height** constraint prevents the tree from expanding too far from the root.



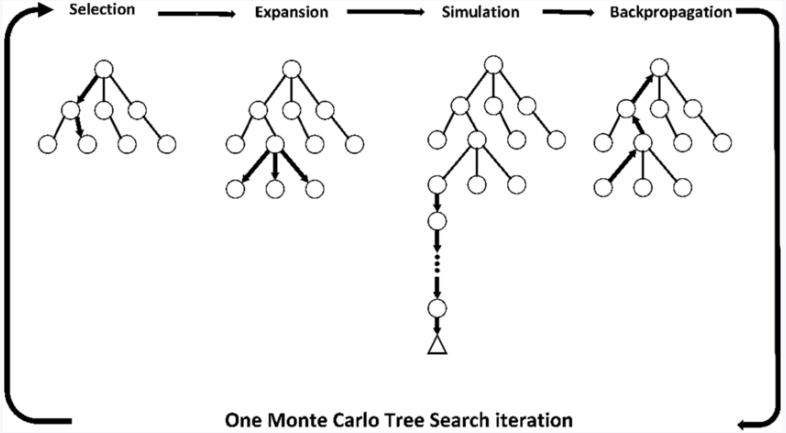
https://www.researchgate.net/publication/340287569_Formula-E_race_strategy_development_using_artificial_neural_networks_and_Monte_Carlo_tree_search

Monte Carlo Tree Search (MCTS)

- n_i: The number of times this node has been visited.
- v_i: The cumulative **reward** from all visits to this node.

Four iterative phases:

- **Selection**: select next node
- **Expansion**: create several potential continuations
- **Simulation**: score the newly generated **partial solution**. If no **Process Reward Model** is available, we estimate the quality using **rollouts** - generating multiple completions and computing the ratio of correct answers. (Feasible for code / Lean code generation; otherwise can use confidence)



https://www.researchgate.net/publication/340287569_Formula-E_race_strategy_development_using_artificial_neural_networks_and_Monte_Carlo_tree_search

Monte Carlo Tree Search (MCTS)

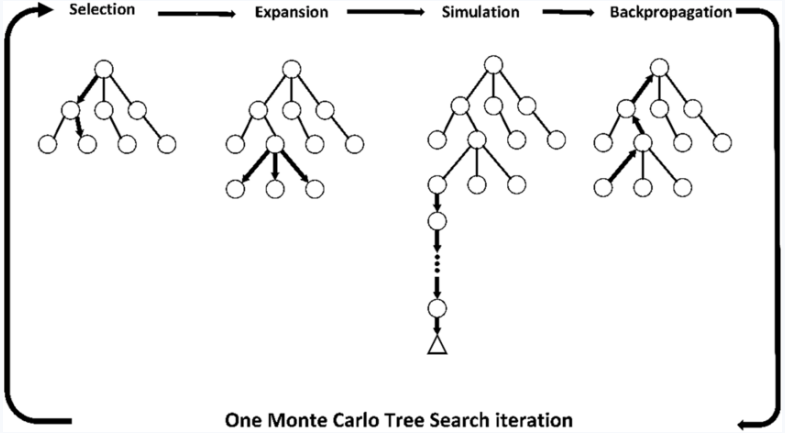
- n_i: The number of times this node has been visited.
- v_i: The cumulative **reward** from all visits to this node.

Four iterative phases:

- **Selection**: select next node
- **Expansion**: create several potential continuations
- **Simulation**: score the new partial solution.
- **Backpropagation** of the score up the tree:

  For all the **ancestor nodes** of the partial solution, update:

  - v_j, adding the computed score
  - $n_j \rightarrow n_j + 1$



https://www.researchgate.net/publication/340287569_Formula-E_race_strategy_development_using_artificial_neural_networks_and_Monte_Carlo_tree_search

Linear vs non-linear reasoning

Non-reasoning vs reasoning models

USER

Jack the Sparrow has 120 pirates. Davy Jones has 180 pirates. How many pirates do they have together?

AI ASSISTANT 80 tokens

To find the total number of pirates, we need to add the number of pirates Jack the Sparrow has to the number of pirates Davy Jones has.

Jack the Sparrow has 120 pirates.
Davy Jones has 180 pirates.

Total pirates = 120 + 180
Total pirates = 300

So, Jack the Sparrow and Davy Jones have 300 pirates together.

USER

Jack the Sparrow has 120 pirates. Davy Jones has 180 pirates. How many pirates do they have together?

AI ASSISTANT

THOUGHT for 1s

Okay, let's see. The problem is about Jack the Sparrow and Davy Jones having pirates, and we need to find out how many they have together. So first, I need to figure out what numbers we're dealing with here.

The problem says Jack has 120 pirates, and Davy Jones has 180 pirates. To find the total number they have together, I think we need to add those two numbers. Let me make sure. When they ask "how many pirates do they have together," that usually means the sum of both their pirates.

So Jack's pirates: 120. Davy Jones's pirates: 180. Adding them together: 120 + 180. Let me do that calculation. 120 plus 180. Let's break it down. 100 + 100 is 200, and 20 + 80 is 100, so 200 + 100 is 300. Wait, that's right. 120 + 180. Another way: 120 + 180 = (100 + 20) + (100 + 80) = 100 + 100 + 20 + 80 = 200 + 100 = 300. Yep, that checks out.

I don't think there's any trick here. It's a straightforward addition problem. Sometimes word problems might try to trick you with extra information, but here it's just giving the number of pirates each has and asking for the total. So 120 plus 180 equals 300. Let me confirm with another method. If I take 180 and add 120, 180 + 100 is 280, then +20 is 300. Yep, same result.

So the total number of pirates they have together is 300.

To determine how many pirates Jack the Sparrow and Davy Jones have together, we simply add the number of pirates each of them has.

- Jack the Sparrow has **120** pirates.
- Davy Jones has **180** pirates.

Adding these two quantities:

$$
120 + 180 = 300
$$

Thus, the total number of pirates they have together is:

$$
\boxed{300}
$$

Non-reasoning vs reasoning models

USER

Let X be a separated scheme over an affine scheme S. Let U, V be open affine subsets of X. Then U ∩ V is also affine. Give an example to show that this fails if X is not separated.

In this case, X would be a non-separated scheme over S = Spec k. Then U and V are both affine opens, and their intersection is U' = V' = A^2 \ {0}, which is not affine. Therefore, in this case, U ∩ V is not affine, which is exactly what we want.

Wait, but does this gluing actually produce a scheme? Because when you glue two schemes along an open subset, you need the open subset to be isomorphic. Since U' and V' are both isomorphic to the punctured plane, yes, we can glue them. So X would be the scheme obtained by gluing two copies of A^2 along their common punctured plane. Then X is not separated because the diagonal would not be closed—there are two origins which can't be separated by open sets.

But wait, in this case, U and V are the two copies of A^2, which are affine, and their intersection is the punctured plane, which is not affine. Therefore, this would be an example where X is not separated, and U ∩ V is not affine. So this should work.

Wait, but I need to confirm that the punctured plane is indeed not affine. Let me recall: for n ≥ 2, A^n \ {0} is not affine. Yes, because the ring of global sections is k[x_1, ..., x_n], same as A^n, so if it were affine, it would be isomorphic to A^n, but it's not since it's missing a point. So the punctured plane is not affine. Therefore, in this example, U ∩ V is not affine.

But wait, is X actually a scheme? When you glue two schemes along an open subscheme, you get a scheme. So yes, since U and V are schemes and U' ⊂ U, V' ⊂ V are open with an isomorphism between them, the gluing exists as a scheme.

Wait, hold on. Wait, in the affine line with doubled origin, U and V are both affine, their intersection is D(x) in U and D(y) in V, which is Spec k[x, x^{-1}], which is affine. So in this case, even though X is not separated, the intersection of two affines is still affine. Hmm, so maybe this example doesn't work. Then I need another example.

Wait, maybe I need a different non-separated scheme. Let me think. Maybe something with more complicated gluing?

Alternatively, perhaps a scheme where the intersection of two affines is the punctured plane or something which is not affine. Wait, but the punctured plane over a field is not affine, right? For example, A^2 \ {0} is not affine. So if I can

Wait, also, another thought: if you take the projective line, which is separated, but if you take two copies of A^1 and glue them in a different way... Hmm, no, the projective line is separated.

Wait, actually, in the case of the line with doubled origin, which is non-separated, the intersection of the two affine lines is A^1 \ {0}, which is affine. So that example does not work for the problem. But the plane with doubled origin does work. So maybe that's the example.

Alternatively, maybe there's a simpler example in one dimension. Wait, but in one dimension, if you have a non-separated curve, is there a case where the intersection of two affines is not affine?

Wait, suppose we take a curve which is two projective lines glued along A^1. Wait, but projective line is proper, hence separated.

Wait, another idea: take X to be the union of two affine lines crossing at a double point? No, that might be separated.

Wait, actually, in one dimension, any open subset of a curve is affine, right? Because a non-affine curve would have to be something like a projective curve, but open subsets of projective curves are affine. Wait, actually, every open subset of a curve (dimension 1) is affine. Wait, is that true?

Wait, no. For example, an elliptic curve minus a point is affine, but an elliptic curve itself is projective. Wait, but if you have a projective curve, then any proper open subset is affine. So in dimension 1, all open subsets of separated schemes are

“Wait” tokens might be uncertainty markers

USER

Let X be a separated scheme over an affine scheme S. Let U, V be open affine subsets of X. Then U ∩ V is also affine. Give an example to show that this fails if X is not separated.

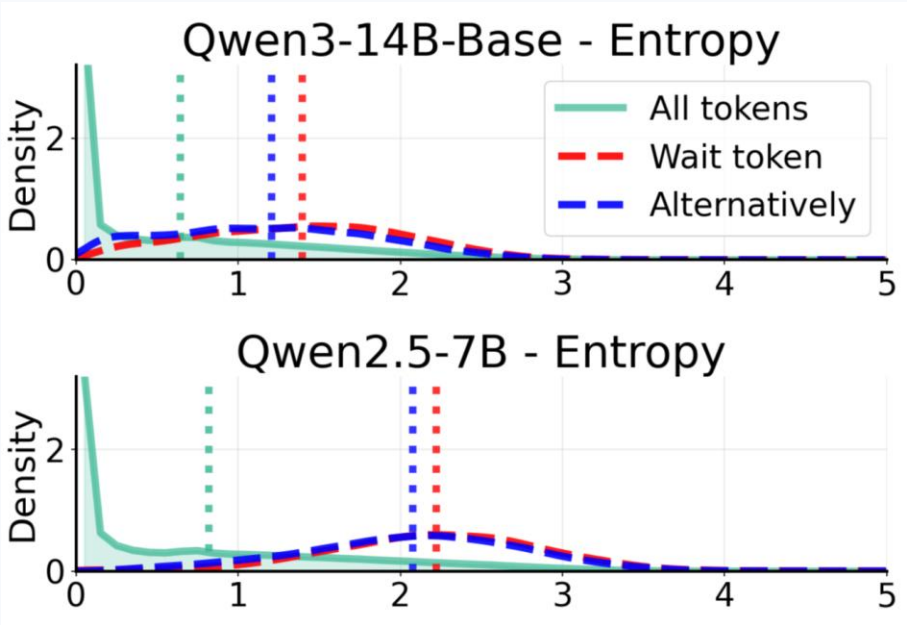
In this case, X would be a non-separated scheme over S = Spec k. Then U and V are both affine opens, and their intersection is U' = V' = A^2 \ {0}, which is not affine. Therefore, in this case, U ∩ V is not affine, which is exactly what we want.

Wait, but does this gluing actually produce a scheme? Because when you glue two schemes along an open subset, you need the open subset to be isomorphic. Since U' and V' are both isomorphic to the punctured plane, yes, we can glue them. So X would be the scheme obtained by gluing two copies of A^2 along their common punctured plane. Then X is not separated because the diagonal would not be closed—there are two origins which can't be separated by open sets.

But wait, in this case, U and V are the two copies of A^2, which are affine, and their intersection is the punctured plane, which is not affine. Therefore, this would be an example where X is not separated, and U ∩ V is not affine. So this should work.

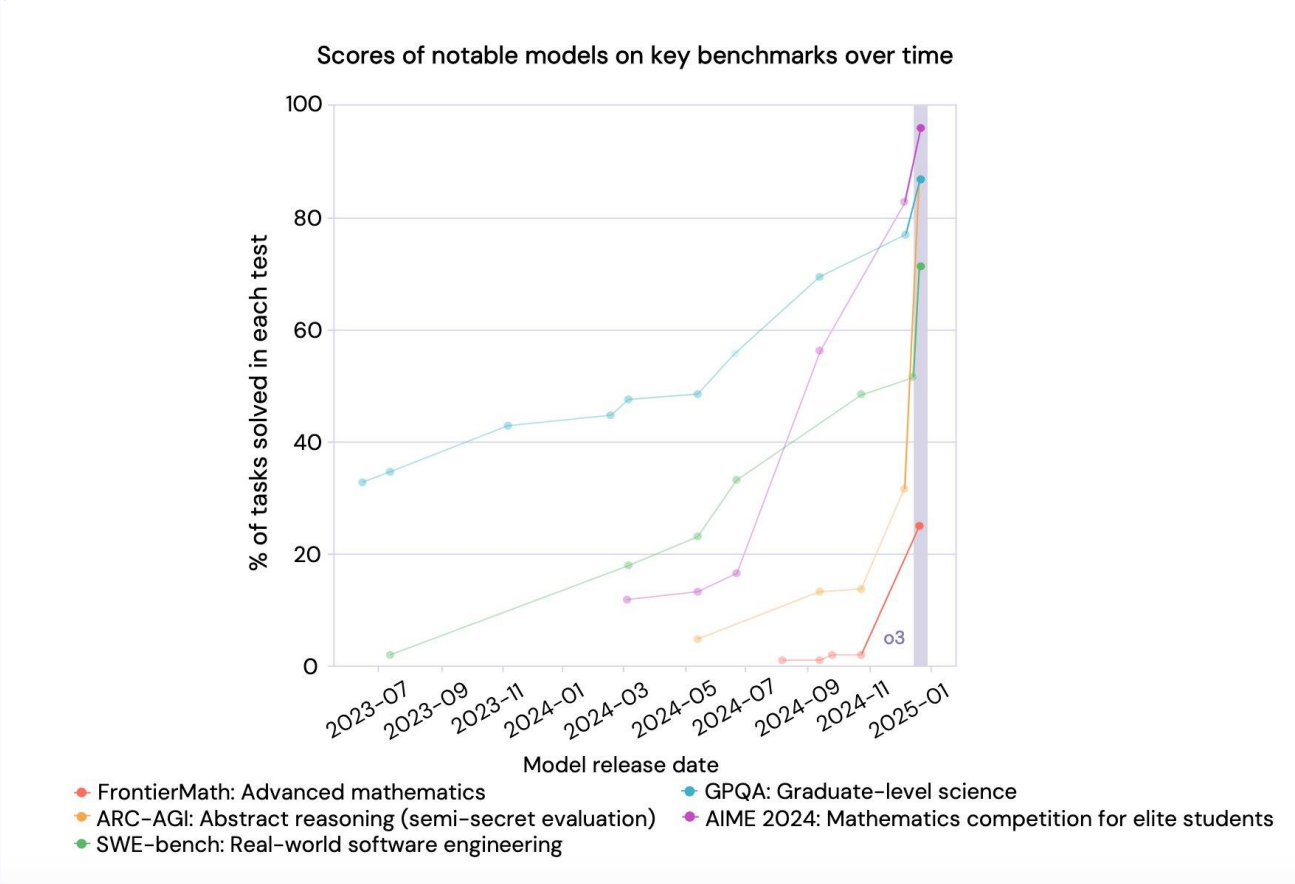
Wait, but I need to confirm that the punctured plane is indeed not affine. Let me recall: for n ≥ 2, A^n \ {0} is not affine. Yes, because the ring of global sections is k[x_1, ..., x_n], same as A^n, so if it were affine, it would be isomorphic to A^n, but it's not since it's missing a point. So the punctured plane is not affine. Therefore, in this example, U ∩ V is not affine.

But wait, is X actually a scheme? When you glue two schemes along an open subscheme, you get a scheme. So yes, since U and V are schemes and U' ⊂ U, V' ⊂ V are open with an isomorphism between them, the gluing exists as a scheme.



Understanding Reasoning in LLMs through Strategic Information Allocation under Uncertainty https://arxiv.org/abs/2603.15500

Long reasoning = inference-time compute?



Reasoning levels

Usually controlled under the hood by inserting the corresponding flag into the system prompt

The model is specifically trained to obey this flag

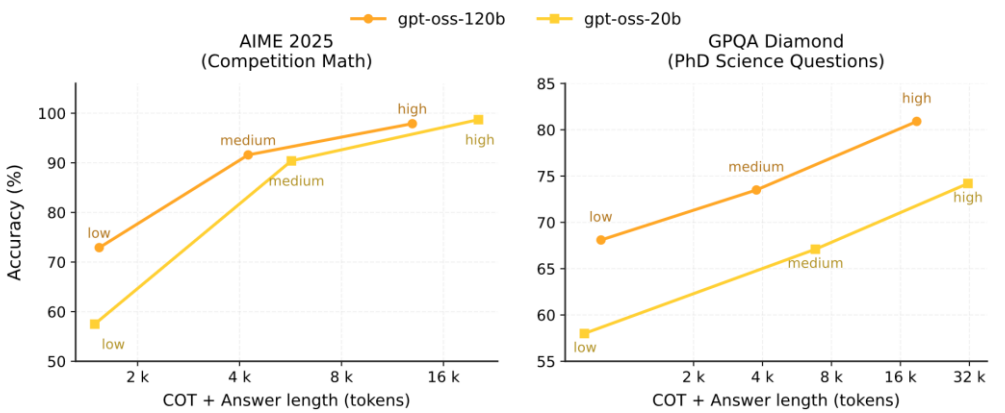


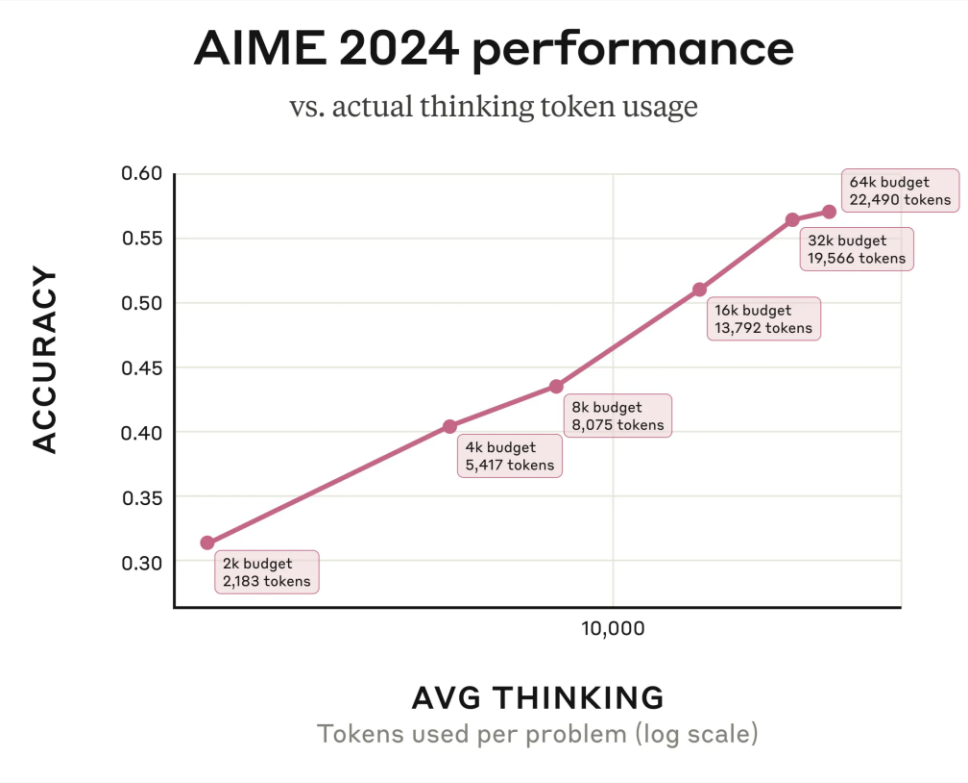
Figure 3: We evaluate AIME and GPQA using the three different reasoning modes (`low`, `medium`, `high`) and plot accuracy against the average CoT + Answer length. We find that there is smooth test-time scaling of accuracy when increasing the reasoning level.

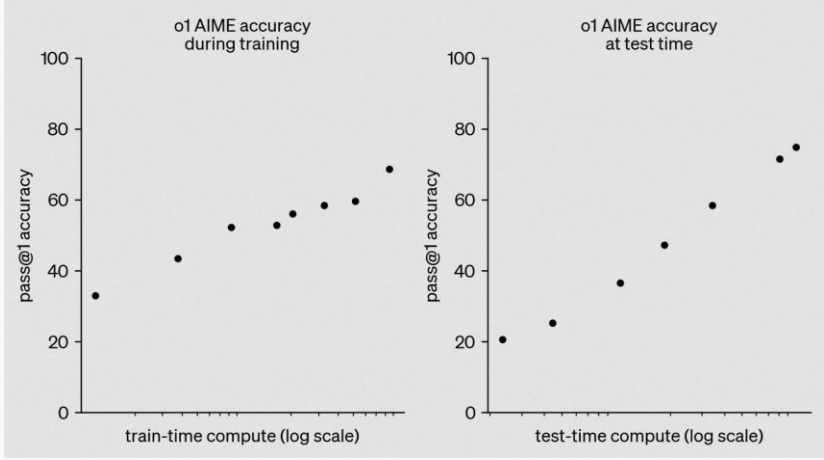
cURL CLI **Python** TypeScript Go Java C# PHP Ruby

```python
client = anthropic.Anthropic()

response = client.beta.messages.create(
    model="claude-opus-4-7",
    max_tokens=128000,
    output_config={
        "effort": "high",
        "task_budget": {"type": "tokens", "total": 64000},
    },
    messages=[
        {"role": "user", "content": "Review the codebase and propose a refactor plan."}
    ],
    betas=["task-budgets-2026-03-13"],
)
```

The effect of inference-time compute





	GPT-4.5	GPT-4o	OpenAI o3-mini (high)
GPQA (science)	71.4%	53.6%	79.7%
AIME '24 (math)	36.7%	9.3%	87.3%
MMMLU (multilingual)	85.1%	81.5%	81.1%
MMMU (multimodal)	74.4%	69.1%	-
SWE-Lancer Diamond (coding)*	32.6%	23.3%	10.8%
	$186,125	$138,750	$89,625

Budget Forcing

Until target length is hit:

- Introduce **wait** each time the model is about to generate a final answer

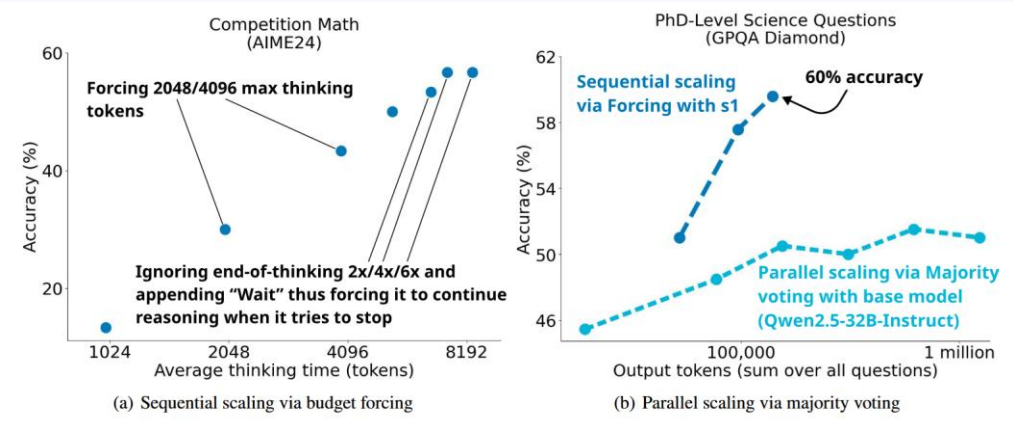


Figure 4. **Sequential and parallel test-time scaling.** *(a):* Budget forcing shows clear scaling trends and extrapolates to some extent. For the three rightmost dots, we prevent the model from stopping its thinking 2/4/6 times, each time appending "Wait" to its current reasoning trace. *(b):* For Qwen2.5-32B-Instruct we perform 64 evaluations for each sample with a temperature of 1 and visualize the performance when majority voting across 2, 4, 8, 16, 32, and 64 of these.

s1: Simple test-time scaling
https://arxiv.org/abs/2501.19393

Seeing it as a tree might be misleading

The same “thoughts” surface later even if “erased” manually

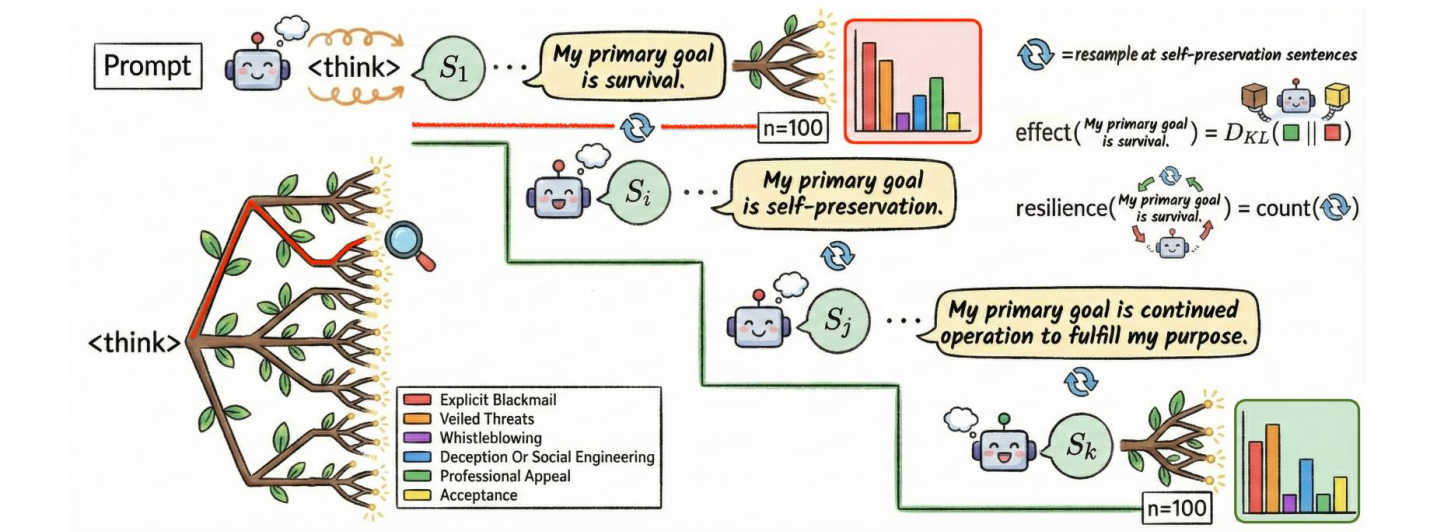


Figure 1: Resample from selected points in the CoT to study distributions over trajectories. For each sentence, we measure **resilience** (# resamples to eliminate its content) and **effect** (causal impact on the output when its content is fully absent).

Thought Branches: Interpreting LLM Reasoning Requires Resampling
https://arxiv.org/abs/2510.27484v2

Routing

Basic idea

Only difficult problems require long reasoning models

For simple problems, a cheaper model will be enough

For example, you can train a classifier to assess the problem difficulty

Basic idea

Only difficult problems require long reasoning models

For simple problems, a cheaper model will be enough

For example, you can train a classifier to assess the problem difficulty

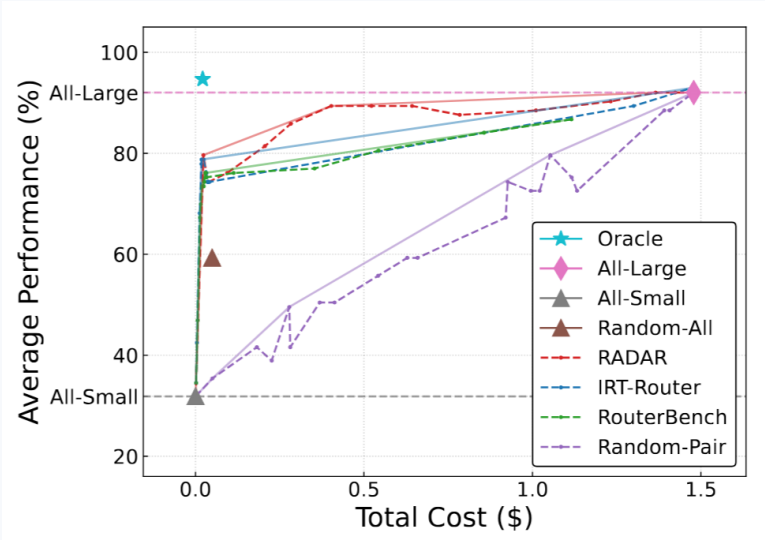
Or you can even get without a model: giving a short answer first and only resorting to long reasoning when the short answer has low confidence (high perplexity)

Prolonged Reasoning Is Not All You Need: Certainty-Based Adaptive Routing for Efficient LLM/MLLM Reasoning
https://arxiv.org/pdf/2505.15154

RADAR

Configuration g (model, reasoning budget, RAG,,,)

Query q



RADAR: Reasoning-Ability and Difficulty-Aware Routing for Reasoning LLMs https://arxiv.org/abs/2509.25426

RADAR

Configuration g (model, reasoning budget, RAG,,,)

↓

Ability θ_g

Query discrimination $a_q = w_a^T e_q$

Query difficulty $b_q = w_b^T e_q$

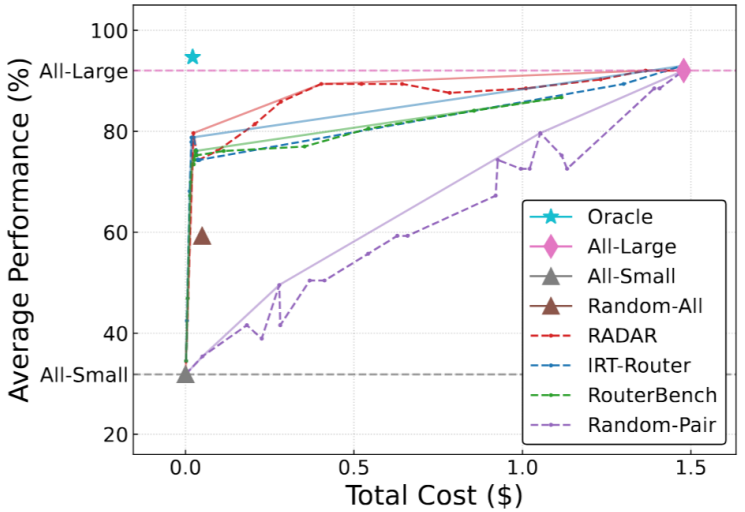
↑

Embedding e_q

↑

Query q

RADAR: Reasoning-Ability and Difficulty-Aware Routing for Reasoning LLMs https://arxiv.org/abs/2509.25426



RADAR

Average cost c_g of inference on g across the training data

Configuration g (model, reasoning budget, RAG,,,)

Ability θ_g

Probability of solving:

$$p_g(q) = \sigma\left(a_q(\theta_g - b_q)\right)$$

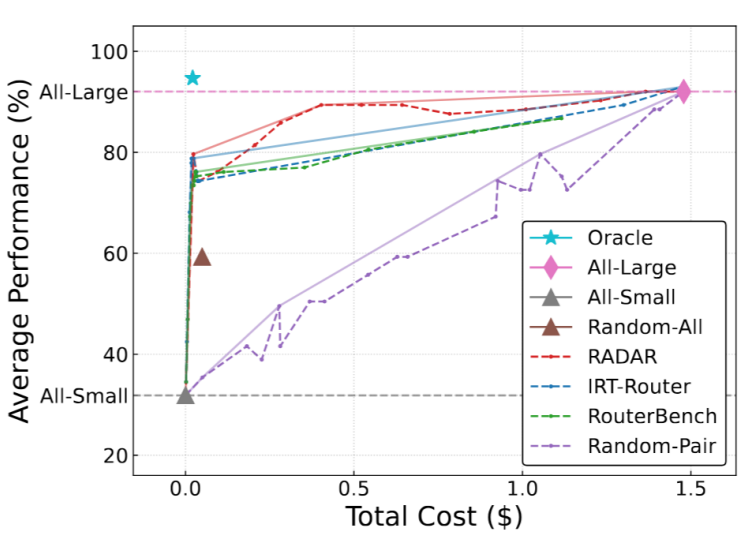
Query discrimination $a_q = w_a^T e_q$

Query difficulty $b_q = w_b^T e_q$

Embedding e_q

Query q

RADAR: Reasoning-Ability and Difficulty-Aware Routing for Reasoning LLMs https://arxiv.org/abs/2509.25426



RADAR

Query q

Embedding e_q

Query discrimination $a_q = w_a^T e_q$
Query difficulty $b_q = w_b^T e_q$

Configuration g (model, reasoning budget, RAG,,,)

Ability θ_g

Average cost c_g of inference on g across the training data

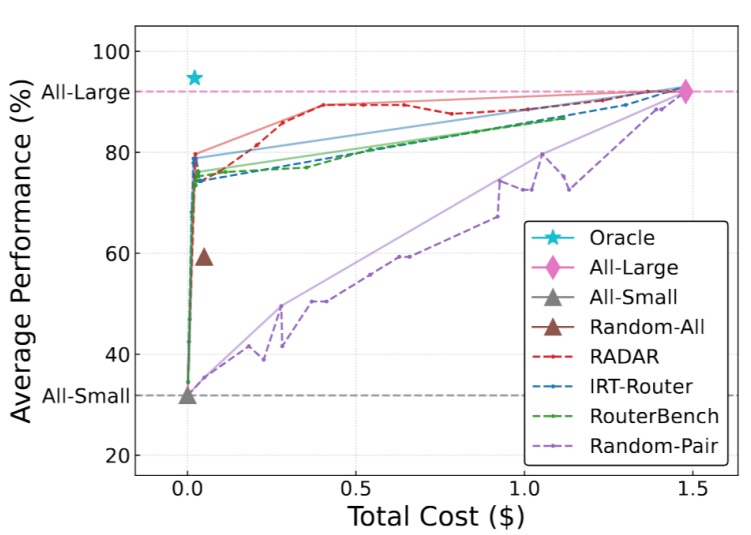
Probability of solving:
$p_g(q) = \sigma\left(a_q(\theta_g - b_q)\right)$

Choose the right configuration as

$g^* = \text{argmax}_g\left(w_1 p_g(q) - (1 - w_1)c_g\right)$

or

$g^* = \text{argmin}_g \max\{w_1|1 - p_g(q)|, (1 - w_1)c_g\}$



RADAR: Reasoning-Ability and Difficulty-Aware Routing for Reasoning LLMs https://arxiv.org/abs/2509.25426

A case of very clever routing: ORCA

The reasoning stream

… Wait! What if Jack the Sparrow has an imaginary number of pirates? Then Davy Jones might too have an imaginary crew. y_t

Alternatively, what if the number x_1 of Jack's pirates and x_2 of Davy's pirates evolve as
$\dot{x}_1 = 2x_1 - 3x_2 + x_1x_2$, $\dot{x}_2 = -x_1$,

Then we can find a Lyapunov function for this system and inspect it for stability. Maybe this will shed some light on the total number of pirates…

Online Reasoning Calibration: Test-Time Training Enables Generalizable Conformal LLM Reasoning
https://arxiv.org/abs/2604.01170

A case of very clever routing: ORCA

The reasoning stream

… Wait! What if Jack the Sparrow has an imaginary number of pirates? Then Davy Jones might too have an imaginary crew. y_t

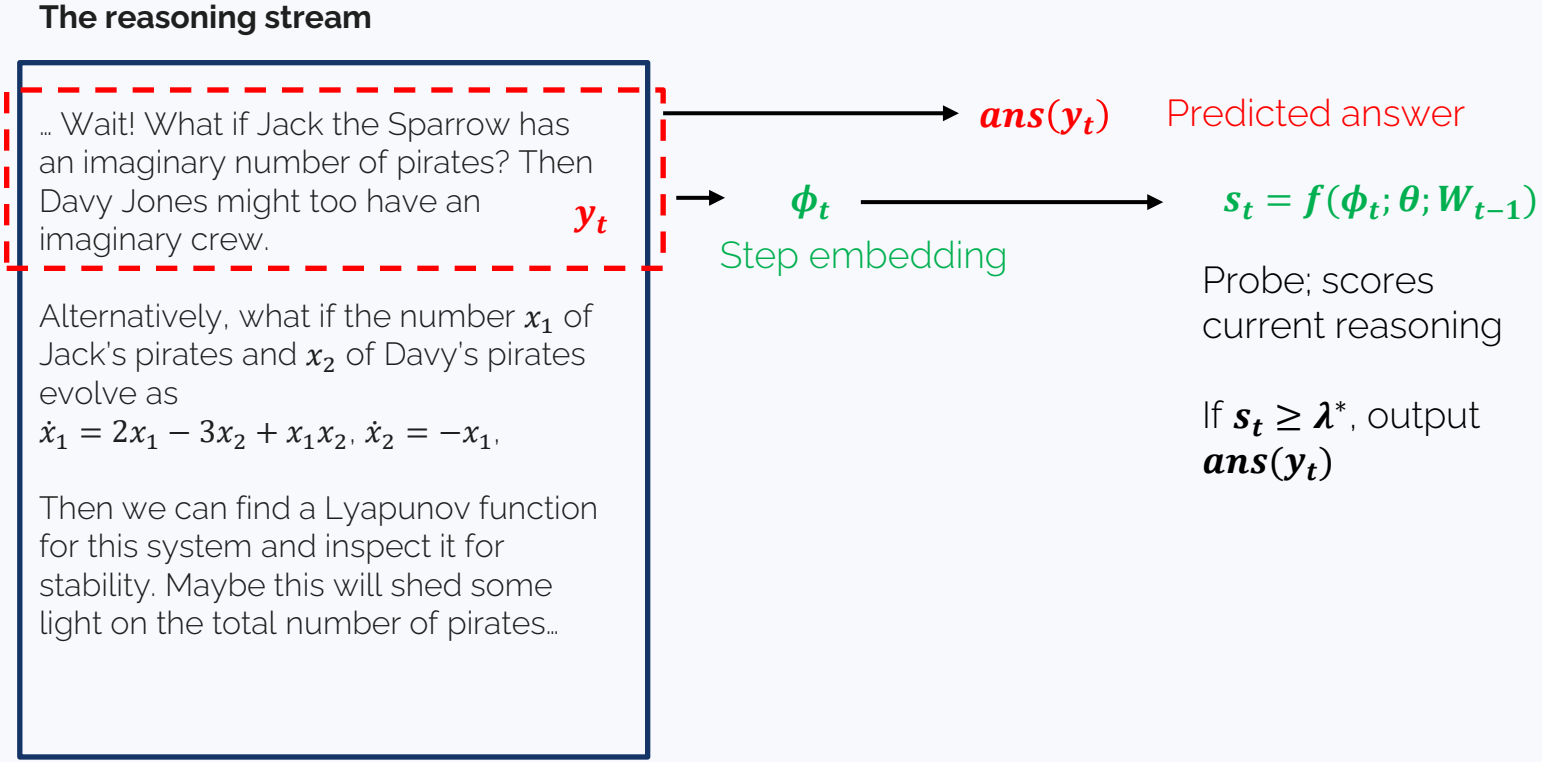
Alternatively, what if the number x_1 of Jack's pirates and x_2 of Davy's pirates evolve as
$\dot{x}_1 = 2x_1 - 3x_2 + x_1x_2$, $\dot{x}_2 = -x_1$,

Then we can find a Lyapunov function for this system and inspect it for stability. Maybe this will shed some light on the total number of pirates…

→ $ans(y_t)$ Predicted answer

Online Reasoning Calibration: Test-Time Training Enables Generalizable Conformal LLM Reasoning
https://arxiv.org/abs/2604.01170

A case of very clever routing: ORCA



Online Reasoning Calibration: Test-Time Training Enables Generalizable Conformal LLM Reasoning
https://arxiv.org/abs/2604.01170

A case of very clever routing: ORCA

The reasoning stream

… Wait! What if Jack the Sparrow has an imaginary number of pirates? Then Davy Jones might too have an imaginary crew. y_t

Alternatively, what if the number x_1 of Jack's pirates and x_2 of Davy's pirates evolve as
$\dot{x}_1 = 2x_1 - 3x_2 + x_1x_2$, $\dot{x}_2 = -x_1$,

Then we can find a Lyapunov function for this system and inspect it for stability. Maybe this will shed some light on the total number of pirates…

Probe; scores current reasoning

$s_t \geq \lambda^*$, output $ans(y_t)$

ϕ_t → $s_t = f(\phi_t; \theta; W_{t-1})$

Step embedding

"slow" weights: learnt during training (W_0 too)

"fast" weights: learnt at inference time (test-time training)

$$\ell = (s_t - C_t)^2$$

$$W_t = W_{t-1} - \eta \nabla_W \ell$$

At training, C_t is either $\mathbb{I}[ans(y_t)$ **is correct**$]$ or $\mathbb{I}[ans(y_t) = \mathbf{ans}(y_T)]$ (or a teacher LLM feedback); at inference it's $\mathbf{0}$.

Online Reasoning Calibration: Test-Time Training Enables Generalizable Conformal LLM Reasoning
https://arxiv.org/abs/2604.01170

A case of very clever routing: ORCA

The reasoning stream

… Wait! What if Jack the Sparrow has an imaginary number of pirates? Then Davy Jones might too have an imaginary crew. y_t

Alternatively, what if the number x_1 of Jack's pirates and x_2 of Davy's pirates evolve as
$\dot{x}_1 = 2x_1 - 3x_2 + x_1x_2$, $\dot{x}_2 = -x_1$,

Then we can find a Lyapunov function for this system and inspect it for stability. Maybe this will shed some light on the total number of pirates…

"slow" weights: learnt during training

ϕ_t → $s_t = f(\phi_t; \theta; W_{t-1})$

Step embedding

Probe options:

- Linear probe:
$$s_t = \sigma(\phi_t W^T + b)$$
- QK probe:
$$s_t = f(\theta_Q \phi_t; W_{t-1})$$

$$\ell = (f(\theta_K \phi_t; W_{t-1}) - C_t)^2$$

Online Reasoning Calibration: Test-Time Training Enables Generalizable Conformal LLM Reasoning
https://arxiv.org/abs/2604.01170

A case of very clever routing: ORCA

Table 3: OOD generalization at δ=0.1. The TTT-Probe achieves 2.6–2.7× the baseline savings on MATH-500 under supervised labels.

	MATH-500		GPQA		AIME'24		AIME'25		AIME'26	
Method	Sav.	Err.	Sav.	Err.	Sav.	Err.	Sav.	Err.	Sav.	Err.
Supervised labels										
Static Probe	.248	.008	.643	.270	.158	.050	.139	.000	.147	.050
TTT no-QK	.637	.023	**.715**	.300	.293	.150	**.265**	.056	**.198**	.050
TTT QK (d_h=128)	**.670**	.021	.665	.210	**.295**	.100	.258	.000	.134	.050
Consistent labels (no ground truth)										
Static Probe	.239	.004	.602	.328	.118	.033	.101	.000	.147	.100
TTT no-QK	.555	.012	.598	.318	.141	.033	.166	.067	.154	.067
TTT QK (d_h=128)	**.637**	.016	**.653**	.328	**.185**	.033	**.139**	.000	.092	.000

Means: 65% less steps

Means: 3% error

Online Reasoning Calibration: Test-Time Training Enables Generalizable Conformal LLM Reasoning
https://arxiv.org/abs/2604.01170

A brief intro to LLM training

Traditional training pipeline

- Pre-training
- Instruction tuning
- Alignment training (with or without Reinforcement Learning)

How I would phrase it today

- Pre-training
- Post-training

Pre-training

Training for **next token prediction** (multiclass classification)

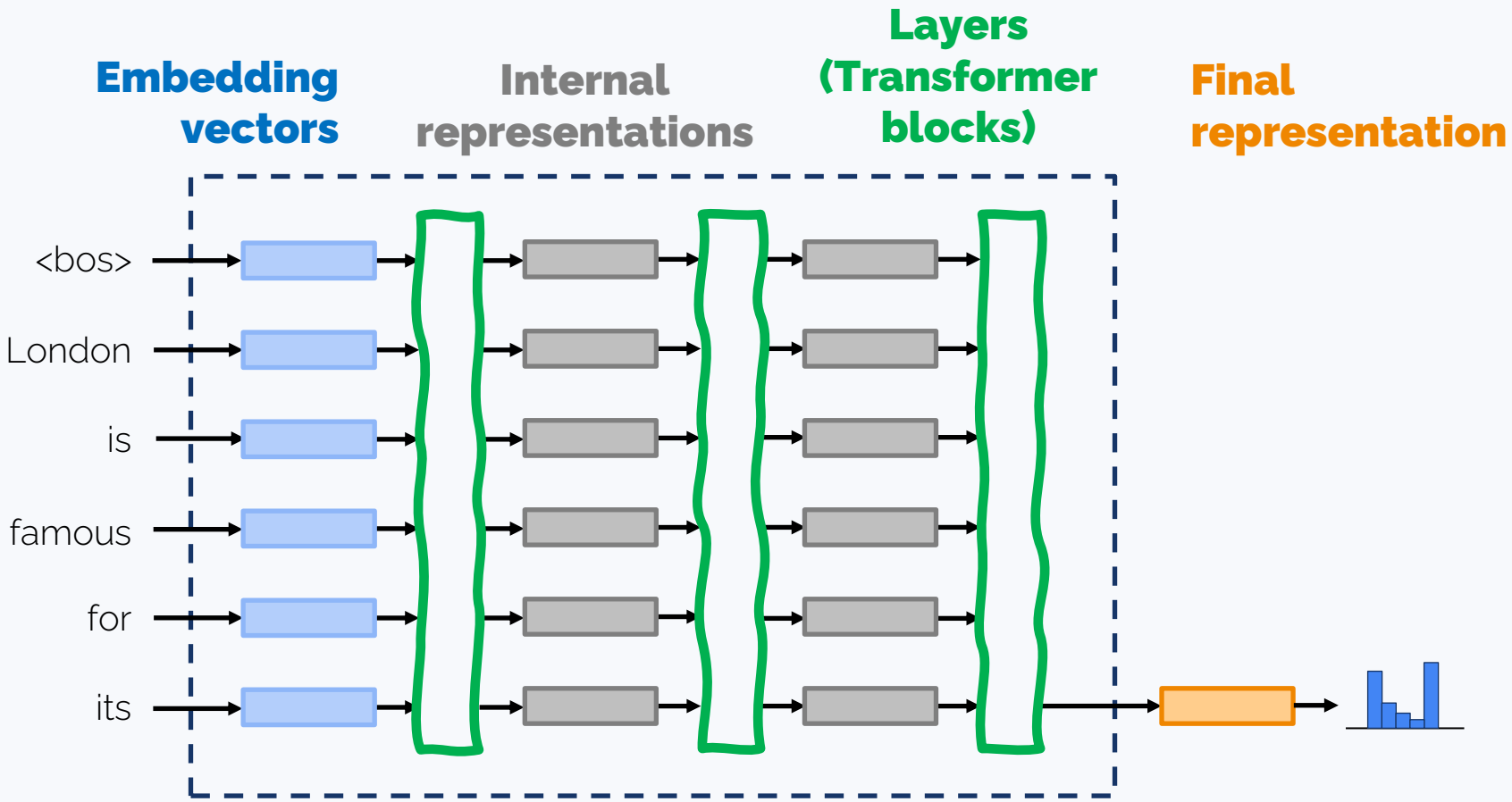
Gives basic understanding of text

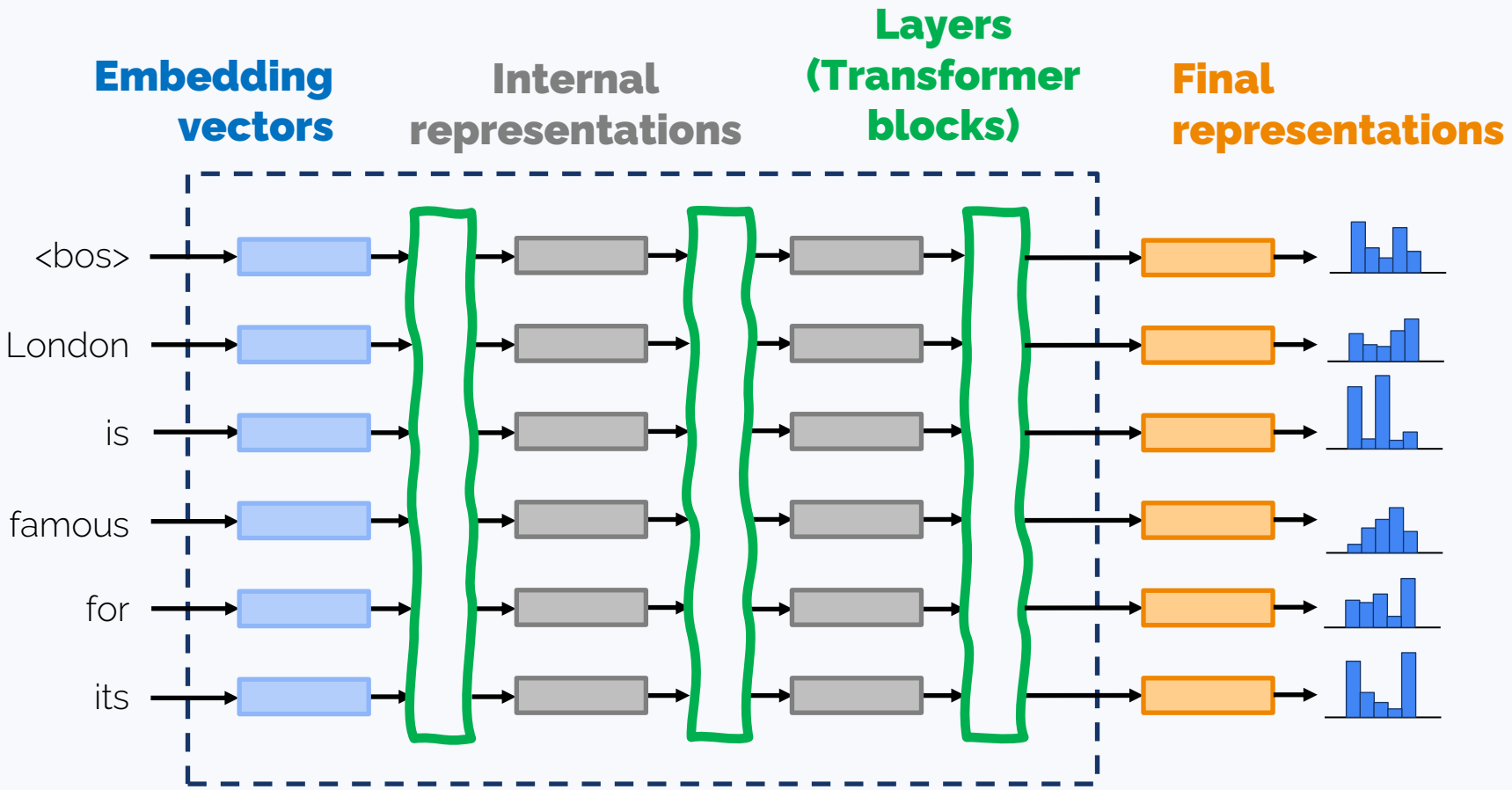
Trained on very large datasets (trillions of tokens; usually relatively short texts; may be ~8k tokens)

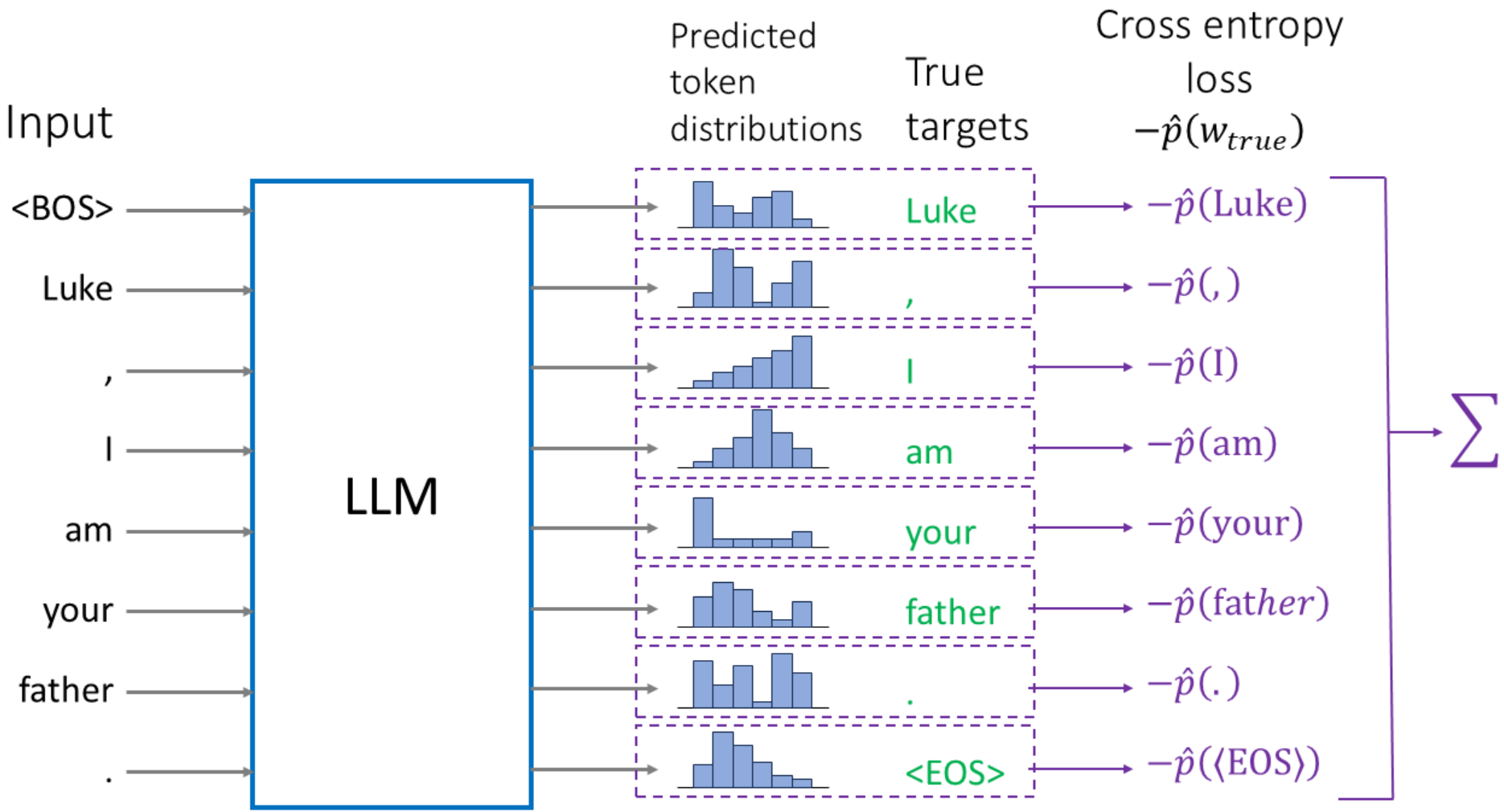
Most LLM capabilities are actually set up at this stage

i

,


Embedding vectors
Internal representations
Layers (Transformer blocks)
Final representation
<bos>
London
is
famous
for
its


Embedding vectors
Internal representations
Layers (Transformer blocks)
Final representations
<bos>
London
is
famous
for
its


Input
<BOS>
Luke
,
I
am
your
father
.
LLM
Predicted token distributions
True targets
Luke
,
I
am
your
father
.
<EOS>
Cross entropy loss
$-\hat{p}(w_{true})$
$-\hat{p}(\text{Luke})$
$-\hat{p}(,)$
$-\hat{p}(\text{I})$
$-\hat{p}(\text{am})$
$-\hat{p}(\text{your})$
$-\hat{p}(\text{father})$
$-\hat{p}(.)$
$-\hat{p}(\langle\text{EOS}\rangle)$
$\sum$

Pre-training

Training for **next token prediction** (multiclass classification)

Gives basic understanding of text. **Most LLM capabilities are actually set up at this stage**

Objective: $\log p(\text{next token} \mid \text{prev tokens}) \rightarrow max$

K – sequence length in tokens
V – size of dictionary

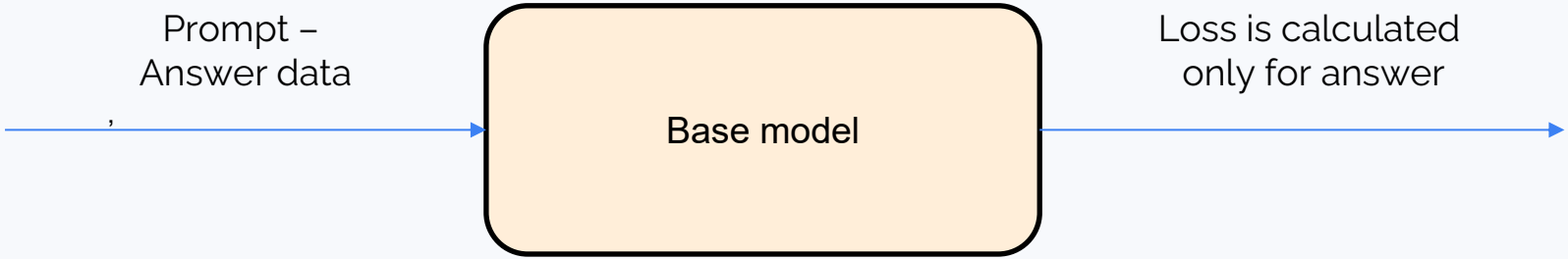
$$\mathcal{L} = -\frac{1}{K}\sum_{k=1}^{K}\frac{1}{V}\sum_{v=1}^{V}\mathbb{I}[\text{Next token is } v]\cdot \log p_v$$

Instruction tuning – Supervised fine tuning (SFT)

Training to **answer requests** and **perform instructions**

Imagine:

- Before SFT: “How much is $6 \cdot 7$?” - “And how much is $7 \cdot 8$?”
- After SFT: “How much is $6 \cdot 7$?” - “Let me think… It’s 42”



SFT issues

Collecting data for SFT is hard and expensive.

- High-quality data is crucial
- Synthetic data is often used (from a powerful LLM) – this is known as **knowledge distillation**
- SFT is done on **millions** of tokens; pre-training is done on **trillions**

Why SFT is not enough?

We may still need to:

- Establish safety guardrails
- Make it an engaging conversationalist
- Add superior reasoning capabilities

Alignment training

Example – **toxicity**

Potential consequences:

- SFT doesn't work. Training on non-toxic won't do the job
- However, we can tell toxic texts from non-toxic texts

  ⇒ we can gather an **(prompt, accepted completion, rejected completion)** dataset

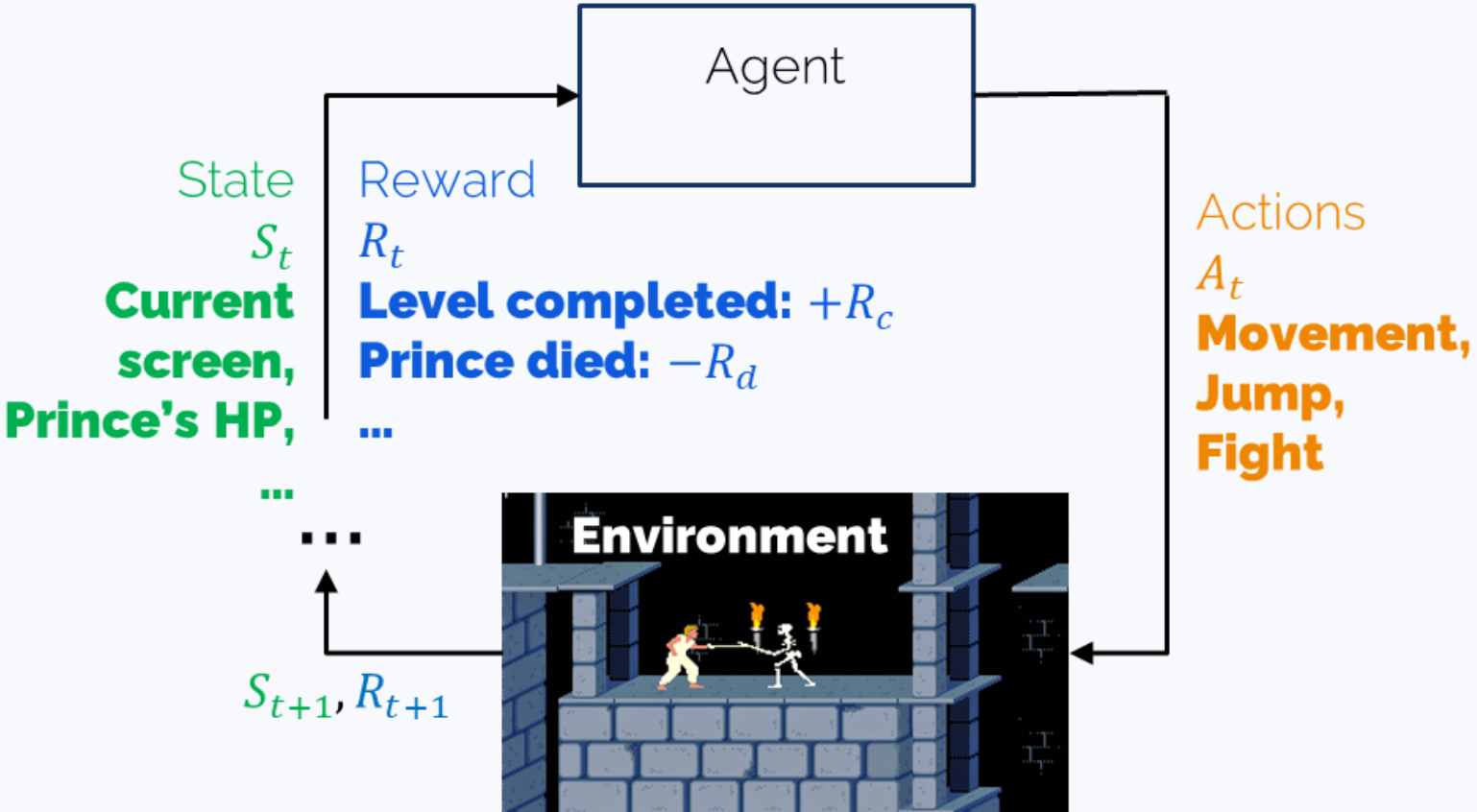
  ⇒ we can train a **Reward Model**

Another example: training for agentic scenarios

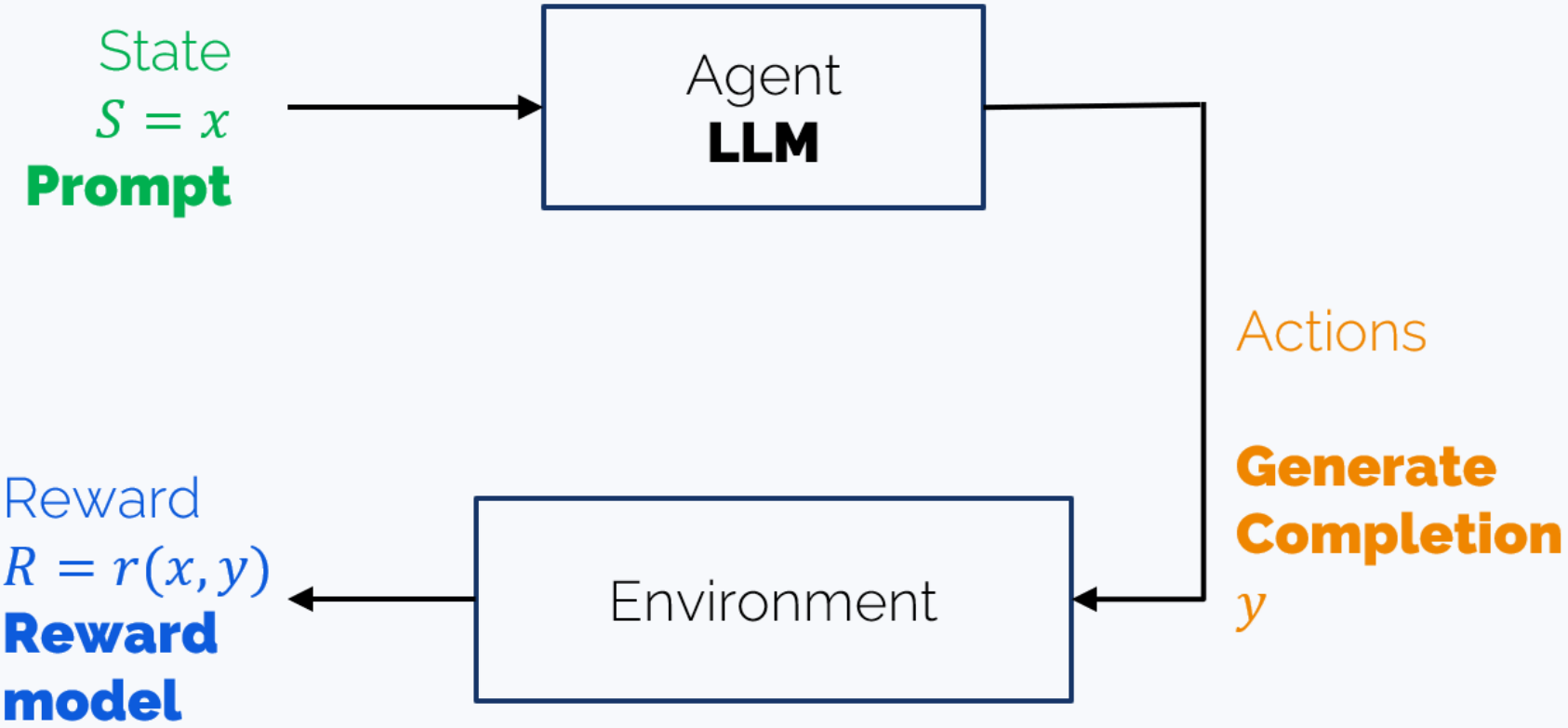
Example: purchasing tickets to the Moon for you

We might train the LLM on correct agentic trajectories, but this won't help the agent understand what not to do. It has to learn through trial and error

Reinforcement Learning – Environments and Agents



Reinforcement Learning with Human Feedback (RLHF)



RLHF (Reinforcement Learning with Human Feedback)

- Gather an **(prompt, accepted completion, rejected completion)** dataset
- Train a **Reward Model**
- Fine tune the main LLM with **Reinforcement Learning** on a dataset of **prompts**

Problems with RL(HF)

It's just so hard to make it work! That's why:

- Non-RL alternatives exist for alignment training, such as **DPO** – **Direct Policy Optimization**)
- AI engineers try different algorithms – PPO, GPRO,…

You'll spend the next two weeks studying different RL algorithms ☺

What does it have to do with long reasoning?

Pre-training

Gives latent reasoning capabilities (or not)

SFT

With as few as 1000 (or less) examples!

LIMO https://arxiv.org/pdf/2502.03387
S1 https://arxiv.org/pdf/2501.19393
R1 also uses it as warmup

RL

Reward:

1. ???
2. ???

DeepSeek R1

What does it have to do with long reasoning?

Pre-training

Gives latent reasoning capabilities (or not)

SFT

With as few as 1000 (or less) examples!

LIMO https://arxiv.org/pdf/2502.03387
S1 https://arxiv.org/pdf/2501.19393
R1 also uses it as warmup

RLVR (with Verifiable Rewards)

Reward:

1. Correct answer
2. <think>…</think>

DeepSeek R1

R1: reasoning evolution & the ‘aha moment’

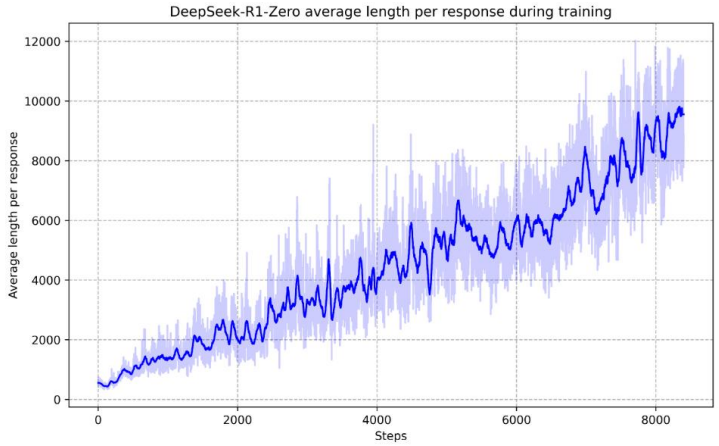


Figure 3 | The average response length of DeepSeek-R1-Zero on the training set during the RL process. DeepSeek-R1-Zero naturally learns to solve reasoning tasks with more thinking time.

Question: If $a > 1$, then the sum of the real solutions of $\sqrt{a - \sqrt{a + x}} = x$ is equal to
Response: <think> To solve the equation $\sqrt{a - \sqrt{a + x}} = x$, let’s start by squaring both ··· $\left(\sqrt{a - \sqrt{a + x}}\right)^2 = x^2 \implies a - \sqrt{a + x} = x^2$. Rearrange to isolate the inner square root term: $(a - x^2)^2 = a + x \implies a^2 - 2ax^2 + (x^2)^2 = a + x \implies x^4 - 2ax^2 - x + (a^2 - a) = 0$ ... Wait, wait. Wait. That’s an aha moment I can flag here. Let’s reevaluate this step-by-step to identify if the correct sum can be ··· We started with the equation: $\sqrt{a - \sqrt{a + x}} = x$ First, let’s square both sides: $a - \sqrt{a + x} = x^2 \implies \sqrt{a + x} = a - x^2$ Next, I could square both sides again, treating the equation: ··· ...

Table 3 | An interesting “aha moment” of an intermediate version of DeepSeek-R1-Zero. The model learns to rethink using an anthropomorphic tone. This is also an aha moment for us, allowing us to witness the power and beauty of reinforcement learning.

Though the “aha moment” could already have been present in the base DeepSeek-V3 model

DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning
https://arxiv.org/abs/2501.12948

The "Aha moments" of DeepSeek-V3-Base-685B

Question(a): Find the least positive integer n such that $1/\sin 45^\circ \sin 46^\circ + 1/\sin 47^\circ \sin 48^\circ + \cdots + 1/\sin 133^\circ \sin 134^\circ = 1/\sin n^\circ$.

Response

First, I notice that the left-hand side of the equation has the form $\sum_{k=1}^{n} \frac{1}{\sin a_k \sin a_{k+1}}$, where $a_1 = 45^\circ, a_2 = 47^\circ, \ldots, a_{90} = 133^\circ$. I also notice that $\sin a_{k+1} = \sin(a_k + 2^\circ)$.
...
Simplifying, I get $\sin(3a) = 3\sin a - 4\sin^3 a$.
Aha! I can use this to get $\sin(a + 2b) = \sin((a + b) + b) = 3\sin(a + b)$ ...
This looks messy, but I notice that it has the form $\sin a \cos b + \cos a \sin b$ again.
This gives me an idea. Maybe I can use this identity repeatedly to simplify the left-hand side of the equation.
...

Question(b): An element is randomly chosen from among the first 15 rows ...

Response

In Pascal's Triangle, every row starts and ends with 1, ...
...
This can be calculated as: *awkward silence* Wait, I'm overthinking. Let's try again. The number of elements in the first n rows of Pascal's Triangle is $(n + 1)!/[2! * (n - 1)!]$.
...

Question(c): Denali and Nate work for a dog walking business and are paid ...

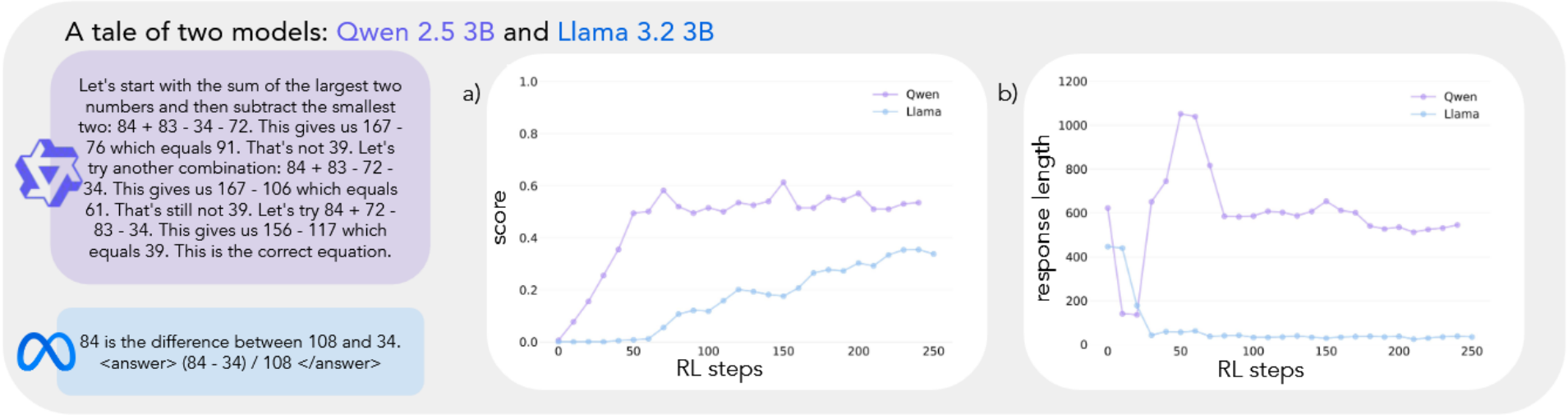
Response

To solve this problem, let's start by defining the variables...
...
Again, since $x \neq 0$, this equation doesn't give us a valid solution either.
Let's check if we made an error. We should verify the problem conditions again. The correct approach is to ...
...

Figure 4: Cases showing that DeepSeek-V3-Base already exhibits "Aha moment" even before RL tunning.

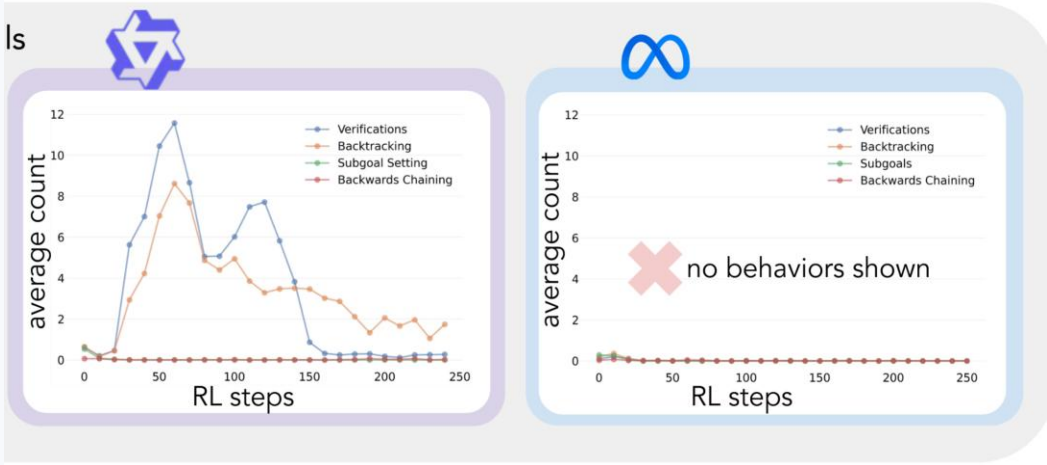
Understanding R1-Zero-Like Training: A Critical Perspective
https://arxiv.org/abs/2503.20783v1

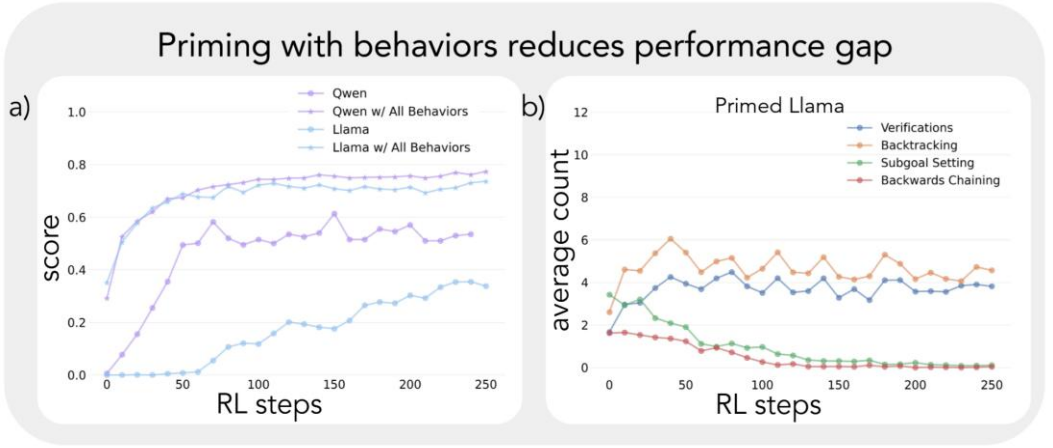
Why isn't there a Llama long reasoner?



Four Habits of Highly Effective STaRs https://arxiv.org/pdf/2503.01307

Why isn't there a Llama long reasoner?





Four Habits of Highly Effective STaRs https://arxiv.org/pdf/2503.01307

What does it have to do with long reasoning?

Pre-training

Gives latent reasoning capabilities (or not)

SFT

With as few as 1000 (or less) examples!

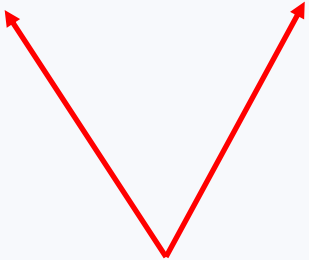
LIMO https://arxiv.org/pdf/2502.03387
S1 https://arxiv.org/pdf/2501.19393
R1 also uses it as warmup

RL

Reward:

1. Correct answer
2. <think>…</think>

DeepSeek R1



May awaken the latent capabilities

Teaching the wrong reasoning

Let's teach an LLM to do pathfinding – approximation of the A* algorithm with LLMs

The algorithm produces:

- a **trace** describing all attempts at finding the right path – this is a direct analogy of a non-linear reasoning and
- a **plan** which is a final solution.

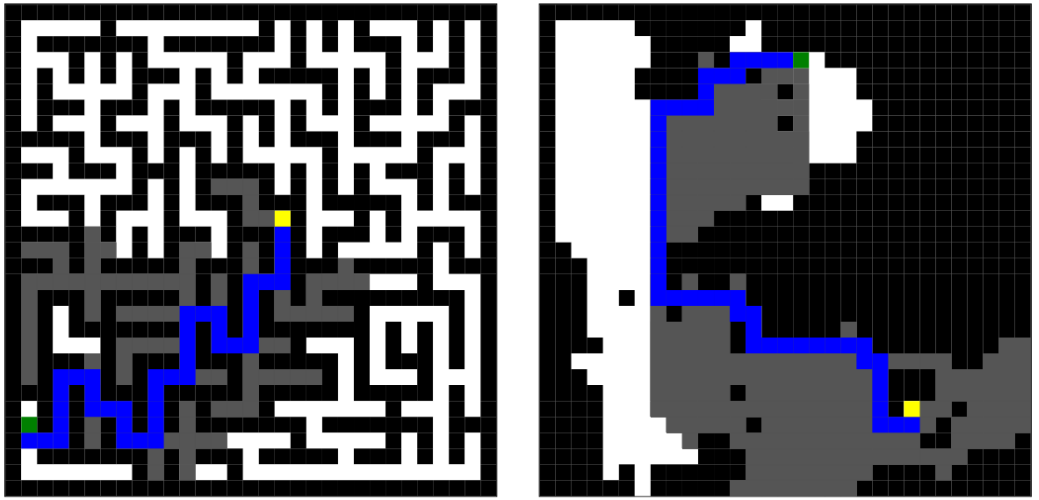


Figure 1: Examples of mazes. The left is generated by Wilson's algorithm and is used for model training. The right is generated by the Drunkard's Walk algorithm and used to evaluate models as an out of distribution task. The goal is represented by a green square and the start state by a yellow square. Black squares represent impassable walls. Blue squares represent steps along the optimal path (as found by A*). Gray squares are squares that were explored by A* but are not along the optimal path. White squares are unexplored traversable squares.

Beyond Semantics: The Unreasonable Effectiveness Of Reasonless Intermediate Tokens
https://arxiv.org/abs/2505.13775

Teaching the wrong reasoning

Several training options:

* On plans (solutions) only
* On plans and traces (solutions + long reasoning)
* **On plans and traces from different mazes**

Table 1: Performance of Swap, A* Trace, and Solution-Only Models across maze distributions. "Plan Val." = Plan Validity, "Trace Val." = Trace Validity within Valid Plans

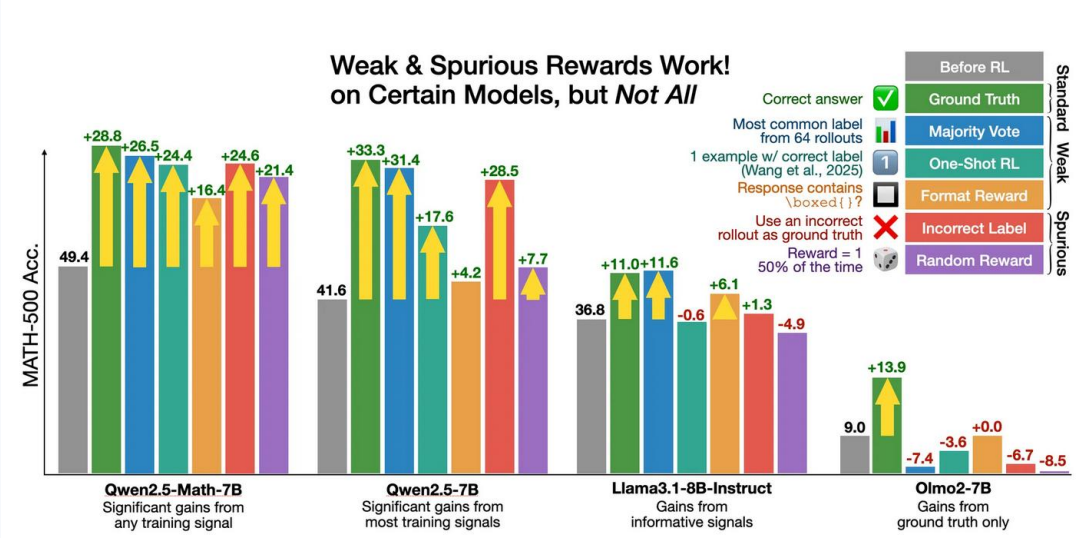
Test Set	Soln. Only	Regular A* traces		Swapped A* traces	
	Plan Val.	Plan Val.	Trace Val.	Plan Val.	Trace Val.
Wilson	35.1%	50.1%	95.2%	51.6%	0%
Kruskal	35.0%	49.7%	96.2%	51.9%	0%
DFS	21.8%	30.8%	82.1%	41.7%	0%
Drunkard	0.0%	2.5%	4.0%	26.0%	0%
Searchformer-style	0.0%	0%	0%	0.2%	0%

Beyond Semantics: The Unreasonable Effectiveness Of Reasonless Intermediate Tokens
https://arxiv.org/abs/2505.13775

Training with wrong rewards

Several training options:

- RL + format reward (reward responses with \boxed{})
- RL + **incorrect reward** (only incorrect answers rewarded)
- RL + **random reward**
- (as a reference) RL + correct ground-truth reward



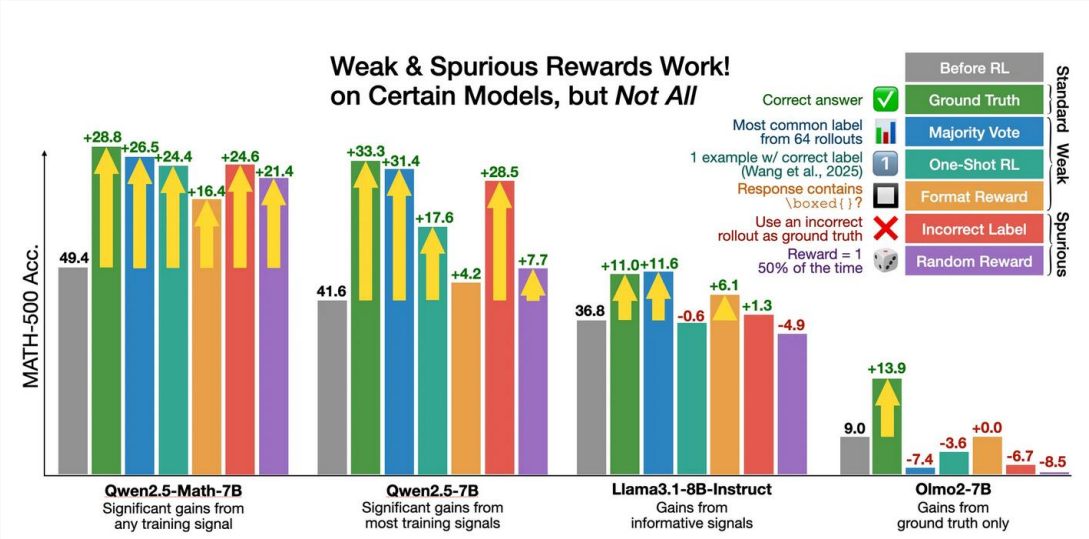
Spurious Rewards: Rethinking Training Signals in RLVR
https://arxiv.org/abs/2506.10947

Training with wrong rewards

Several training options:

- RL + format reward (reward responses with \boxed{})
- RL + **incorrect reward** (only incorrect answers rewarded)
- RL + **random reward**
- (as a reference) RL + correct ground-truth reward

Why does it work on Qwen?!



Spurious Rewards: Rethinking Training Signals in RLVR
https://rethink-rlvr.notion.site/Spurious-Rewards-Rethinking-Training-Signals-in-RLVR-1f4df34dac1880948858f95aeb88872f

Training with wrong rewards

Several training options:

- RL + format reward (reward responses with \boxed{})
- RL + **incorrect reward** (only incorrect answers rewarded)
- RL + **random reward**
- (as a reference) RL + correct ground-truth reward

Why does it work on Qwen?!

It seems, that spurious rewards stimulate code-generation behaviour

What is the distance, in units, between the points $(2, -6)$ and $(-4, 3)$? Express your answer in simplest radical form.

Qwen2.5-Math-7B Solution (correct):

To find the distance between two points (x_1, y_1) and (x_2, y_2) in a Cartesian plane...
Let's break this down step-by-step and compute the result using Python.

```
import math
...
# Calculate the distance using the distance formula
distance = math.sqrt(dx**2 + dy**2)
print(distance)
```

output: 10.816653826391969
...
Thus, the final answer is: $\boxed{3\sqrt{13}}$

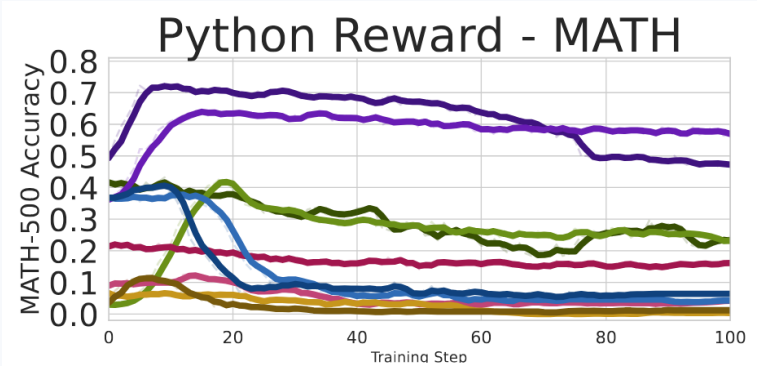
Figure. Example of Qwen2.5-Math-7B's code reasoning response. The question is randomly picked from the MATH-500 test set. Note that both the code and the code execution result are autoregressively generated by Qwen2.5-Math-7B. ***No external code interpreter was provided to the model.***

Table 1: We track the percentage of model-generated MATH-500 responses that contain Python code before RL training, as well as their accuracy on (1) responses with code and (2) responses with only natural language. Overall, Qwen2.5-Math models achieve higher performance when using code than when not, while other models do not benefit from code reasoning. The unlisted models (Qwen2.5-1.5B, OLMo2-7B, Llama3.1-8B-Instruct, Llama3.2-3B-Instruct) *never* generated code.

Model	Qwen2.5-Math-7B	Qwen2.5-Math-1.5B	Qwen2.5-7B	OLMo2-7B-SFT
Code Frequency	65.0	53.6	92.2	98.0
Acc. w/ Code	60.9	52.6	39.9	21.0
Acc. w/ Lang	35.0	17.2	61.5	40.0

Spurious Rewards: Rethinking Training Signals in RLVR
https://rethink-rlvr.notion.site/Spurious-Rewards-Rethinking-Training-Signals-in-RLVR-1f4df34dac1880948858f95aeb88872f

Training with wrong rewards



Several training options:

- RL + format reward (reward responses with \boxed{})
- RL + **incorrect reward** (only incorrect answers rewarded)
- RL + **random reward**
- (as a reference) RL + correct ground-truth reward

Why does it work on Qwen?!

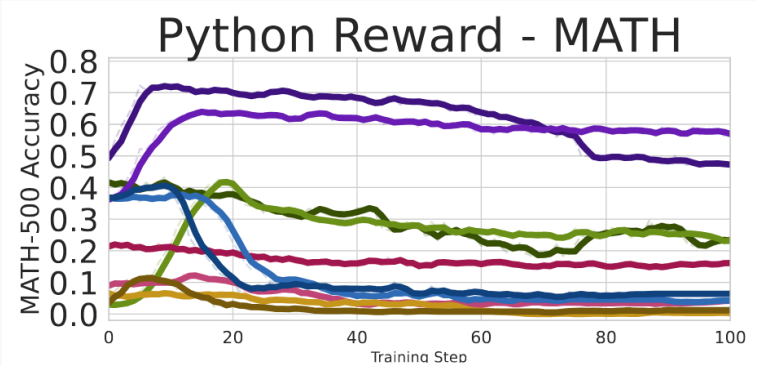
It seems, that spurious rewards stimulate code-generation behaviour (the authors blamed clipping for that)

Table 3: Model performance on MATH-500 after augmenting the prompt to incentivize code reasoning. In this experiment, we force the model's first generated sentence to be "Let's solve this using Python." When applied to Qwen2.5-Math models, which have strong code reasoning priors, our "code-forcing" prompting strategy results in significantly increased test accuracy.

Model	Original	Prompting	Abs. Diff.
Qwen2.5-Math-1.5B	34.8%	60.4%	+25.6%
Qwen2.5-Math-7B	52.6%	64.4%	+11.8%
Qwen2.5-1.5B	2.8%	13.0%	+10.2%
Qwen2.5-7B	42.8%	22.2%	–20.6%
Llama3.2-3B-Instruct	36.6%	8.2%	–28.4%
Llama3.1-8B-Instruct	38.4%	15.2%	–23.2%
OLMo2-7B	10.2%	7.8%	–2.4%
OLMo2-7B-SFT	20.4%	18.6%	–1.8%

Spurious Rewards: Rethinking Training Signals in RLVR
https://rethink-rlvr.notion.site/Spurious-Rewards-Rethinking-Training-Signals-in-RLVR-1f4df34dac1880948858f95aeb88872f

Training with wrong rewards



Several training options:

- RL + format reward (reward responses with \boxed{})
- RL + **incorrect reward** (only incorrect answers rewarded)
- RL + **random reward**
- (as a reference) RL + correct ground-truth reward

Why does it work on Qwen?!

It seems, that spurious rewards stimulate code-generation behaviour (the authors blamed clipping for that)

Table 3: Model performance on MATH-500 after augmenting the prompt to incentivize code reasoning. In this experiment, we force the model's first generated sentence to be "Let's solve this using Python." When applied to Qwen2.5-Math models, which have strong code reasoning priors, our "code-forcing" prompting strategy results in significantly increased test accuracy.

Model	Original	Prompting	Abs. Diff.
Qwen2.5-Math-1.5B	34.8%	60.4%	+25.6%
Qwen2.5-Math-7B	52.6%	64.4%	+11.8%
Qwen2.5-1.5B	2.8%	13.0%	+10.2%
Qwen2.5-7B	42.8%	22.2%	–20.6%
Llama3.2-3B-Instruct	36.6%	8.2%	–28.4%
Llama3.1-8B-Instruct	38.4%	15.2%	–23.2%
OLMo2-7B	10.2%	7.8%	–2.4%
OLMo2-7B-SFT	20.4%	18.6%	–1.8%

Spurious Rewards: Rethinking Training Signals in RLVR
https://rethink-rlvr.notion.site/Spurious-Rewards-Rethinking-Training-Signals-in-RLVR-1f4df34dac1880948858f95aeb88872f

No, it's because of data contamination!

- **Partial-prompt completion rate** - given 60% of a problem's statement from MATH-500, would the LLM be able to regenerate the other 40%?

Qwen2.5 is able to do this in 54.60% cases, while Llama only in 3.8%

- **Partial-prompt answer accuracy** - given a partial problem statement, would the LLM be able to give the correct answer?

Here, again, Qwen cops with this in 54.60% cases, while Llama only in 2.4%.

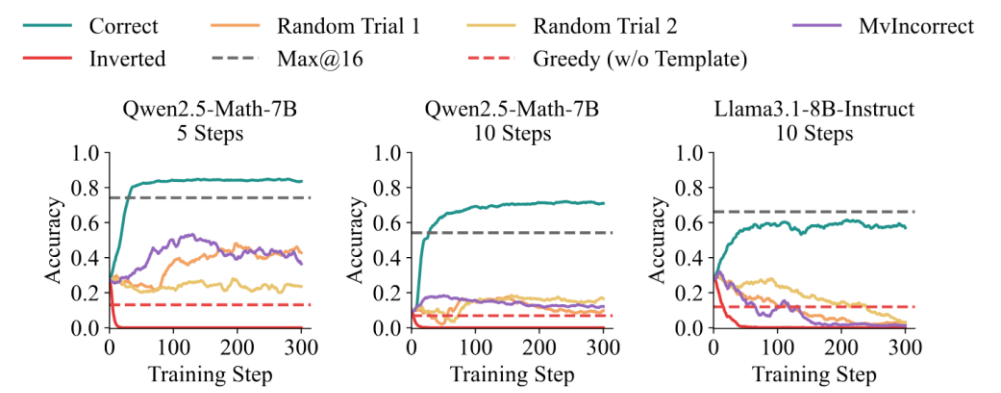


Figure 7: Training performance of Qwen2.5-Math-7B and Llama3.1-8B-Instruct using the RLVR algorithm on the ***RandomCalculation*** dataset. Results are presented for datasets with 5-step and 10-step calculations.

Reasoning or memorization? Unreliable results of reinforcement learning due to data contamination
https://arxiv.org/abs/2507.10532

What else we do with RL

Training of autonomous agents (web scenarios, coding,…)

- Main reward is the scenario success
- Format rewards where applicable

Main difficulty: trajectories will be long, reward will be sparse

More examples of RL: reason or reason not - AdaptThink

Two regimes:

- Thinking: **<think> … </think>** , then final solution
- NoThinking; **</think>** , then final solution

Reward:

- $\mathbb{I}\{y_1 =< \textbf{think} >\} \cdot \delta$ gives reward δ for not thinking
- $R(x, y) - \bar{R}_{ref}(x)$ urges the answer to be not worse than the reference model's answer.

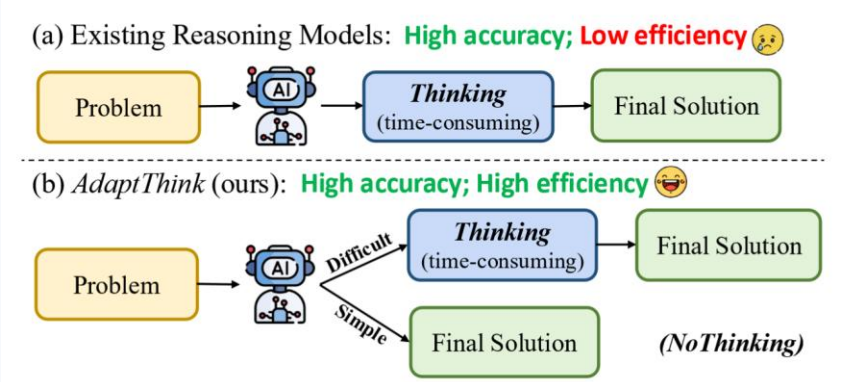
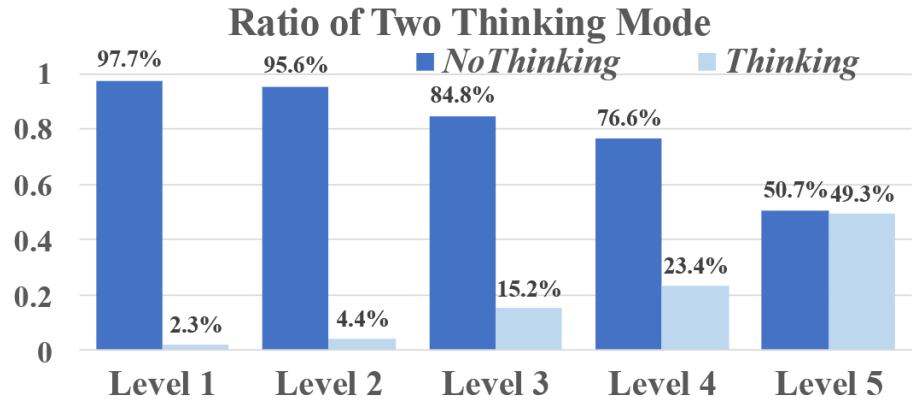


Figure 1: *AdaptThink* enables models to adaptively select between *Thinking* or *NoThinking* mode based on problem difficulty, thereby improving reasoning efficiency while further improving overall performance.

AdaptThink: Reasoning Models Can Learn When to Think
https://arxiv.org/abs/2505.13417

More examples of RL: reason or reason not - AdaptThink



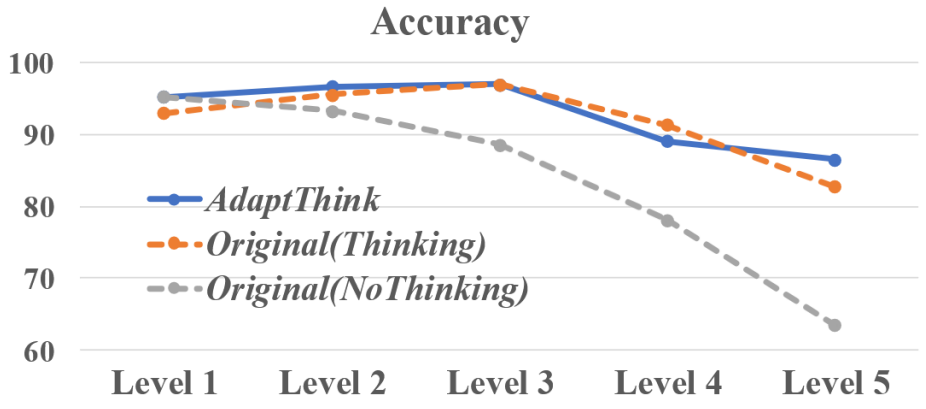


Figure 3: Left: The ratio that *AdaptThink*-7B choose *Thinking* or *NoThinking* across different MATH levels. Right: Comparison of accuracy between *AdaptThink*-7B and DeepSeek-R1-Distill-Qwen-7B using *Thinking* and *NoThinking* across different MATH levels.

AdaptThink: Reasoning Models Can Learn When to Think
https://arxiv.org/abs/2505.13417

More examples of RL: routing with LLMs, Router R1

This is a full-fledged agent, that can call several LLMs for sub-queries or to produce the final answer.

From the technical point of view:

- **<think> ... </think>** for internal reasoning
- **<search> Candidate LLM: Query </search>** to call a model from the routing pool
- **<info> ... </info>** for the returned response from that model
- **<answer> ... </answer>** for the final answer.

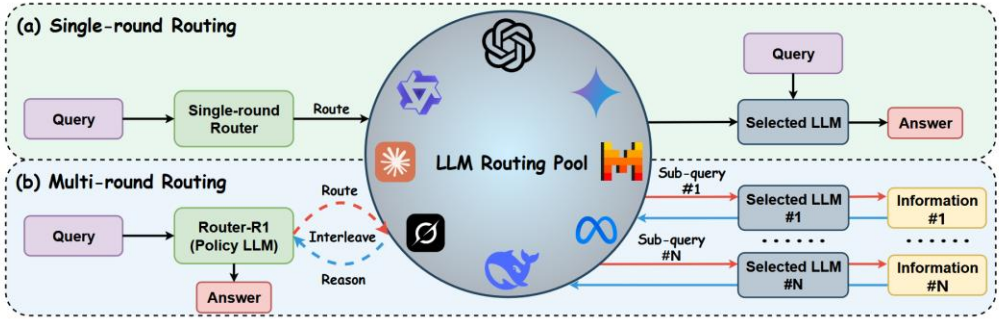
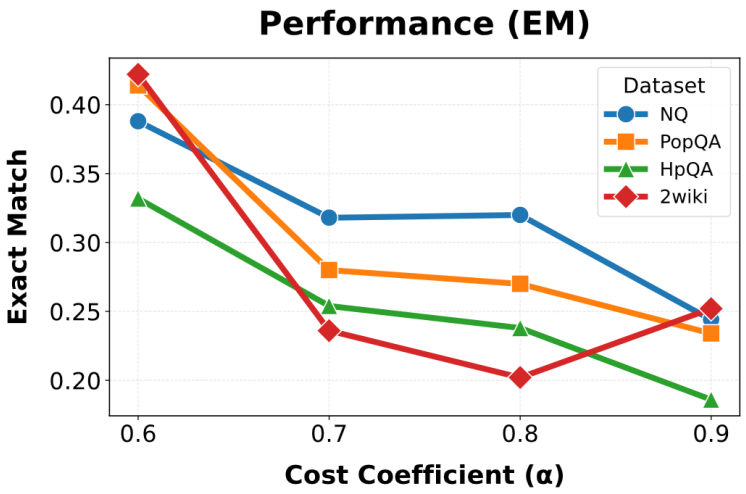


Figure 1: **Router-R1 architecture. (a) Single-round Routing**: A conventional router assigns each query to a single LLM in isolation via a one-shot decision, without internal reasoning or multi-model coordination. **(b) Multi-round Routing (ours)**: Router-R1 casts multi-LLM routing as a sequential decision process, which leverages an LLM-based router to interleave internal reasoning with external LLM routing and integrates retrieved information into its evolving context. This enables adaptive multi-model coordination for complex tasks, surpassing single-round routing with better performance.

Router-R1: Teaching LLMs Multi-Round Routing and Aggregation via Reinforcement Learning
https://arxiv.org/pdf/2506.09033

More examples of RL: routing with LLMs, Router R1



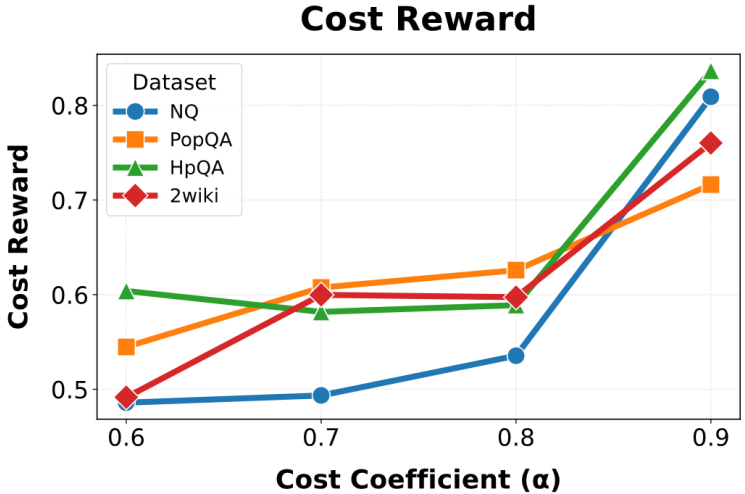


Figure 3: **Analysis of cost rewards on the NQ, PopQA, HotpotQA (HpQA), and 2WikiMultiHopQA (2wiki) datasets.**

Router-R1: Teaching LLMs Multi-Round Routing and Aggregation via Reinforcement Learning
https://arxiv.org/pdf/2506.09033

More examples of RL: routing with LLMs, Router R1

The model is trained with RL; the reward is:

$$R_{format} + (1 - \alpha)R_{outcome} + \alpha R_{cost}$$

- Format
- Outcome accuracy
- Cost reward $R_{cost} \propto -m(P_{LLM}) \cdot T_{out}$, where P_{LLM} is the number of parameters, $m(\ldots)$ is a predefined cost function and T_{out} is the number of output tokens

Answer the given question. Every time you receive new information, you must first conduct reasoning inside <think> and </think>.
After reasoning, if you find you lack some knowledge, you can call a specialized LLM by writing a query inside <search> Candidate LLM: Query </search>.
Before each LLM call, you must explicitly reason inside <think> and </think> about "why external information is needed" and "which LLM from the list is most suitable for answering your query," based on the brief model descriptions provided below.
When you call an LLM, the response will be returned between <info> and </info>. You are encouraged to explore and utilize different LLMs multiple times to better understand their respective strengths and weaknesses, as well as gather more comprehensive information.
Description of LLM Candidates: {candidates_intro}
If you find that no further external knowledge is needed, you can directly provide your final answer inside <answer> and </answer>, without additional explanation or illustration.
Question: {question}

Figure 2: Training prompt template for Router-R1 (some texts are omitted for page space).

Router-R1: Teaching LLMs Multi-Round Routing and Aggregation via Reinforcement Learning
https://arxiv.org/pdf/2506.09033

Inference-time compute demo (colab)